

C3 AI System Design Playbook

Contents

C3 AI System Design Interview Master Doc (Detailed)	1
Table of Contents	2
0. Interview Framework (Deep)	2
1. Parking Lot (C3 actual) — Full Playbook	3
2. Car Rental (Enterprise) (C3 actual) — Full Playbook	12
3. Metrics Logging & Aggregation (C3 actual) — Full Playbook	16
4. Pastebin / Viral Text (C3 actual) — Full Playbook	19
5. Elevator System (C3 OOD) — Full Playbook	20
6. Ticket / Event Booking (Ticketmaster pattern)	24
7. Ride-Sharing	24
8. Dropbox-like File Storage	30
9. URL Shortener	31
10. Rate Limiter / Hit Counter	32
11. Enterprise RAG / Agent Platform	32
12. IoT / Telemetry Ingestion	35
13. Notification System	36
14. Chat / Messaging	37
15. Distributed Job / Workflow System	38
16. Feature Store / Model Serving	38
17. Cross-Cutting Patterns	38
18. Capacity Estimation Workbook	41
19. Failure Modes Catalog	42
20. Practice Rubric & Mocks	42

C3 AI System Design Interview Master Doc (Detailed)

C3 AI design rounds skew **LLD-heavy** (schemas, APIs, race conditions) with occasional **HLD** (Kafka, caching, scale). Confirmed asks: Parking Lot, Car Rental (Enterprise), Metrics Logging, Pastebin-style spikes, Elevator OOD.

Companion files: `interview_stuff.md` · `text.md`

How to use this doc 1. Learn the **§0 framework** cold (first 8 minutes of every interview). 2. For each prompt: redraw the **architecture + ERD + one sequence** on the board while talking. 3. Always close with **concurrency + one failure mode + one scale lever**.

Table of Contents

0. Interview framework (deep)
1. Parking Lot
2. Car Rental
3. Metrics Platform
4. Pastebin / Viral Text
5. Elevator OOD
6. Ticket Booking
7. Ride-Sharing
8. Dropbox-like Storage
9. URL Shortener
10. Rate Limiter
11. Enterprise RAG / Agents
12. IoT Telemetry
13. Notifications
14. Chat / Messaging
15. Job / Workflow Queue
16. Feature Store / Model Serving
17. Cross-cutting patterns
18. Capacity estimation
19. Failure modes
20. Practice rubric

0. Interview Framework (Deep)

0.1 Minute-by-minute script (45 min)

Clock	Goal	Concrete output on board
0:00–1:00	Opening	“Clarify → NFRs → model → APIs → hardest consistency → scale.”
1:00–5:00	Clarify	Actors, scale numbers, consistency needs, v1 scope
5:00–8:00	NFRs	4 bullets with rough SLOs (latency/availability/consistency)
8:00–12:00	Scope	In-scope list + explicit out-of-scope
12:00–22:00	Data model	5–7 entities, PKs/FKs, 2–3 indexes, status enums
22:00–28:00	APIs	5 endpoints with JSON request/response
28:00–38:00	Deep dive	Transaction/lock OR queue/cache path + sequence diagram
38:00–43:00	Scale & failures	One bottleneck + one failure + mitigation
43:00–45:00	Wrap	Trade-offs in 3 sentences

0.2 Clarifying question bank (pick 6–8)

Product - Who are the actors? (end user, operator, device, admin, partner system) - Core user journeys for v1? What can wait for v2? - Single tenant or multi-tenant / multi-region?

Traffic & data - DAU / peak QPS? Read:write ratio? - Object sizes? Retention? Growth per day? - Spiky (viral, batch reconnect) or smooth?

Consistency & UX - Which actions must be exactly-once / strongly consistent? - Is stale search OK for 5–30s? - Idempotent retries expected from mobile clients?

Ops - Compliance (PII, SOC2, HIPAA)? Audit logs needed? - SLO/SLA? RPO/RTO if they care about DR?

0.3 NFR template (fill on board)

```
Latency:    p95 < ___ ms for ___ API
Throughput: ___ peak QPS
Consistency: strong on ___; eventual on ___
Availability: ___ % (e.g. 99.9)
Durability: no loss for ___; best-effort for ___
Security:   authn ___; authz ___; encryption ___
```

0.4 Decision tree: LLD vs HLD

0.5 Consistency cheat sheet

Need	Tool	Say this
One scarce row	SELECT ... FOR UPDATE	"Serialize writers on the spot/vehicle/seat."
High contention, fail-fast	Optimistic version CAS	"Loser gets 409 and retries."
Time-range uniqueness	Exclusion / overlap check	"No two active bookings with overlapping ranges."
Abandoned carts	Hold + TTL worker	"Soft reservation expires in 10 minutes."
Client double-submit	Idempotency-Key	"Unique constraint on key; return first result."
Cross-service	Outbox + webhook reconcile	"Payment and DB can diverge briefly; reconciler fixes."

0.6 Generic layered architecture (HLD backbone)

1. Parking Lot (C3 actual) — Full Playbook

1.1 Problem restatement (say this)

"Drivers find and reserve a spot (optionally by type), enter/exit a lot, and pay. The hard invariant is that one spot is never assigned to two active sessions."

1.2 Clarify (ask out loud)

1. One lot or many lots / floors / gates?
2. Spot types: compact, large, EV, handicap?
3. Walk-in only, or advance reservation?
4. Pricing: flat, hourly, progressive? Payments in-app or at gate?
5. Scale: spots per lot (~500–5000)? Concurrent entries/min?
6. Need admin dashboards / occupancy analytics?

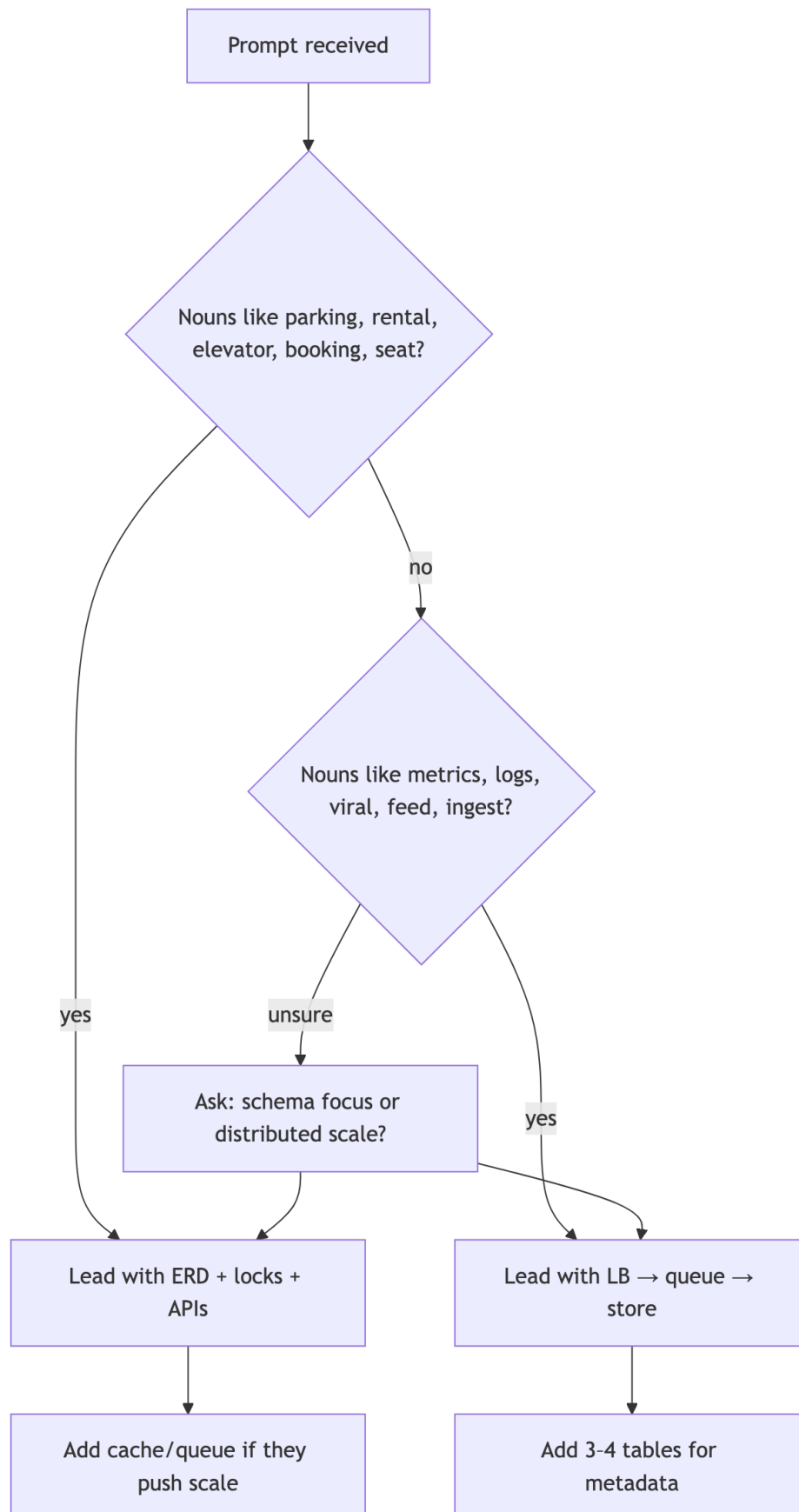


Figure 1: Diagram 1

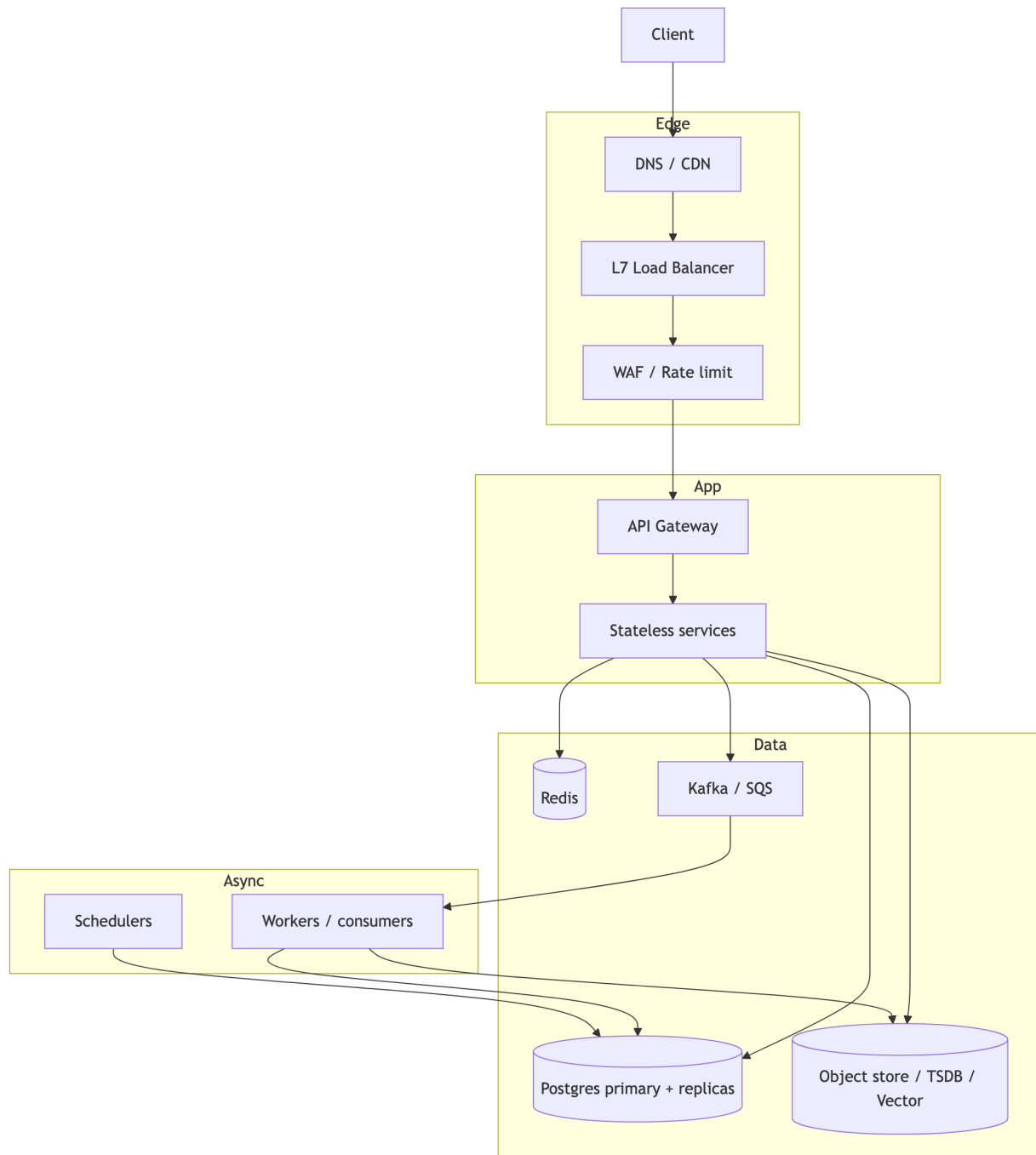


Figure 2: Diagram 2

1.3 Assumptions for v1 (write them)

- Multi-floor lot, typed spots, advance hold + walk-in
- Strong consistency on assign; availability counts may lag 30s
- Card payment via Stripe-like provider
- Out of scope v2: dynamic pricing ML, ANPR cameras, valet

1.4 NFRs (with numbers)

NFR	Target
Assign / check-in p95	< 200 ms
Availability read p95	< 100 ms (cached)
Consistency	Spot status strongly consistent
Availability	99.9% for booking API
Hold TTL	10 minutes
Peak	50 assigns/sec per large lot (burst)

1.5 High-level architecture

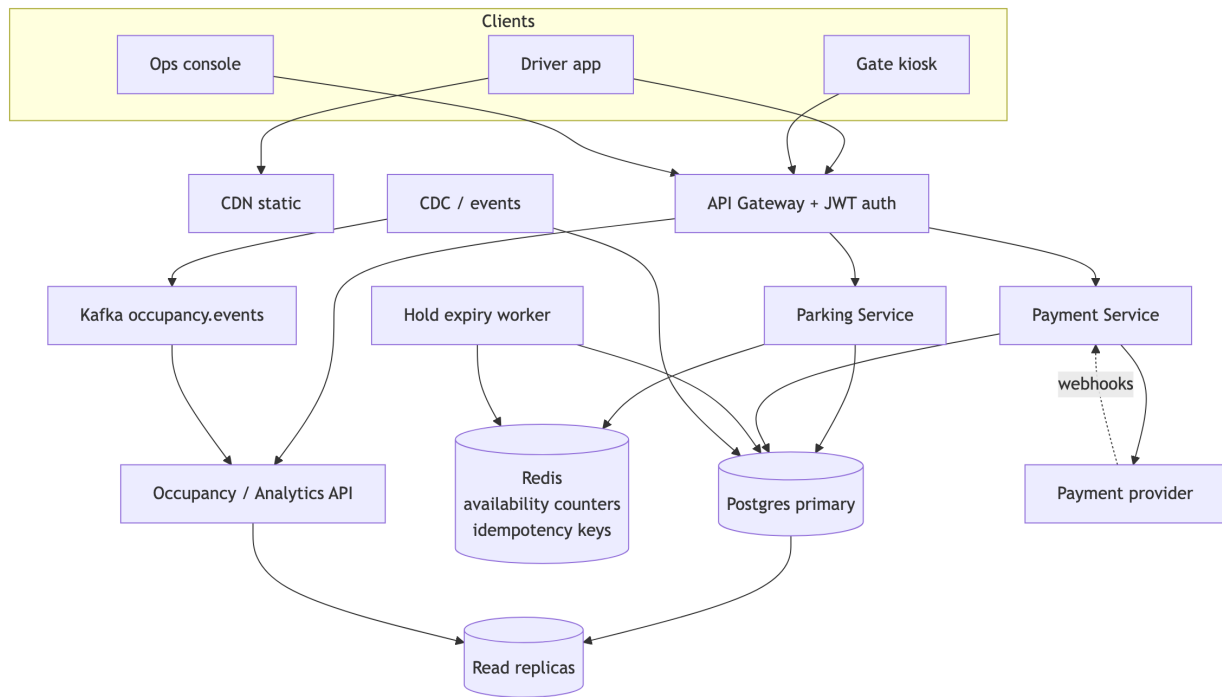


Figure 3: Diagram 3

1.6 Component responsibilities

Component	Responsibility
Parking Service	Availability, holds, check-in/out, spot CAS
Payment Service	Intent/capture, webhook handling, refunds
Redis	Hot free-count by (lot_id, type); idempotency

Component	Responsibility
Postgres	Source of truth for spots & reservations
Expiry worker	held → expired when hold_expires_at < now()
Analytics	Near-real-time occupancy from events (eventual)

1.7 ERD (detailed)

1.8 Indexes

```
CREATE INDEX ON parking_spot (lot_id, type, status);
CREATE UNIQUE INDEX ON parking_spot (lot_id, code);
CREATE INDEX ON reservation (spot_id, status, start_ts);
CREATE UNIQUE INDEX ON reservation (idempotency_key);
CREATE INDEX ON reservation (status, hold_expires_at); -- expiry worker
```

1.9 State machine

1.10 APIs (full JSON)

Create hold

```
POST /v1/reservations
Idempotency-Key: 7f3c...
{ "lotId": "...", "vehicleId": "...", "spotType": "ev", "startTs": "...", "endTs": "..." }

201
{ "reservationId": "...", "spotId": "...", "spotCode": "B-214",
  "status": "held", "holdExpiresAt": "...", "pricingEstimateCents": 1200 }
```

Check-in / out

```
POST /v1/reservations/{id}/check-in → { "sessionId", "entryTs" }
POST /v1/reservations/{id}/check-out → { "feeCents", "paymentStatus" }
```

Availability

```
GET /v1/lots/{lotId}/availability?type=ev
→ { "free": 12, "held": 3, "occupied": 40, "asOf": "...", "cached": true }
```

1.11 Concurrency deep dive (draw this sequence)

Why SKIP LOCKED: under burst, waiters don't pile up on the same row; they take the next free spot.

Optimistic alternative (mention):

```
UPDATE parking_spot SET status='held', version=version+1
WHERE id=$1 AND status='free' AND version=$2;
-- rowcount 0 → conflict → retry pick
```

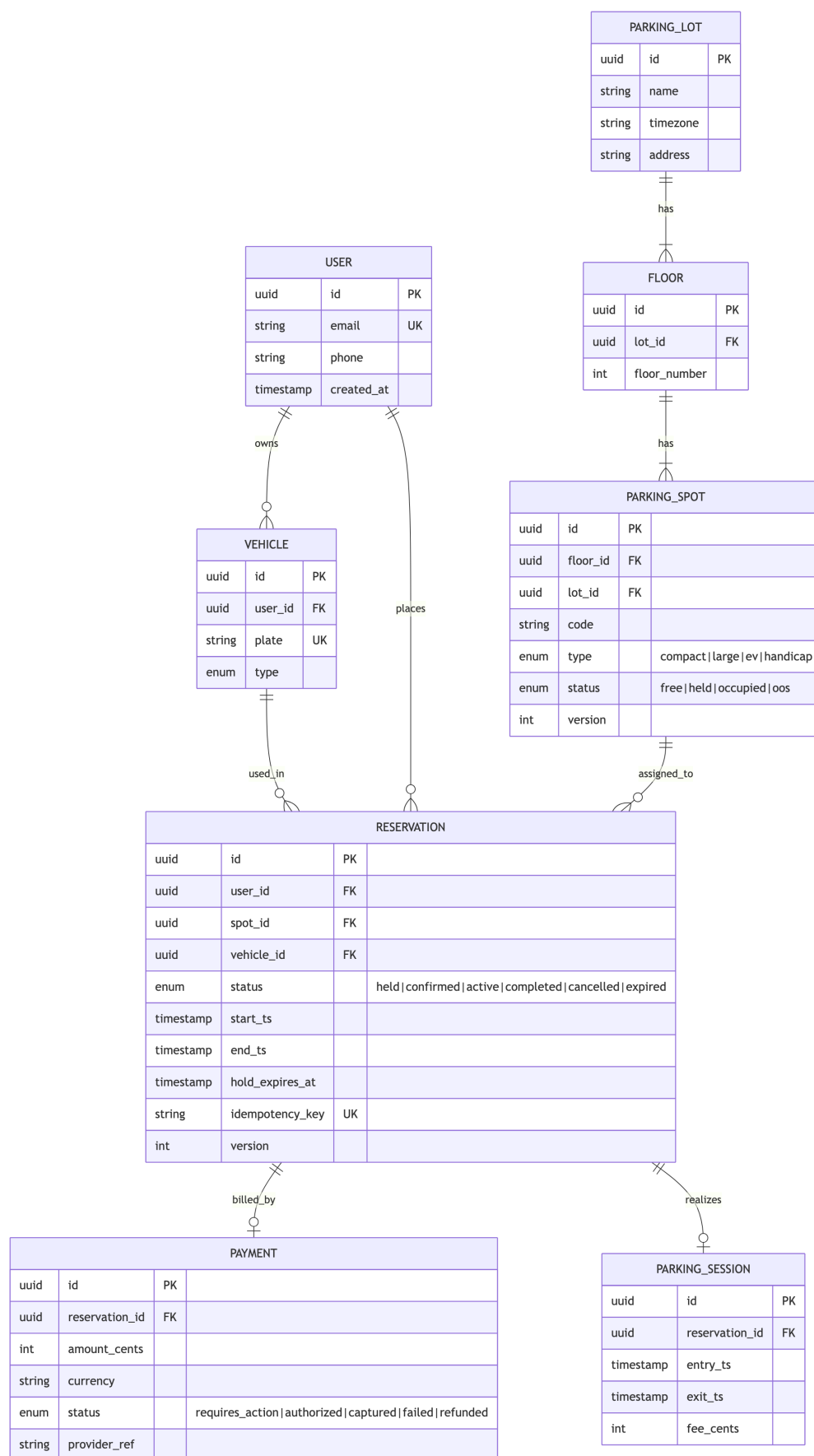


Figure 4: Diagram 4

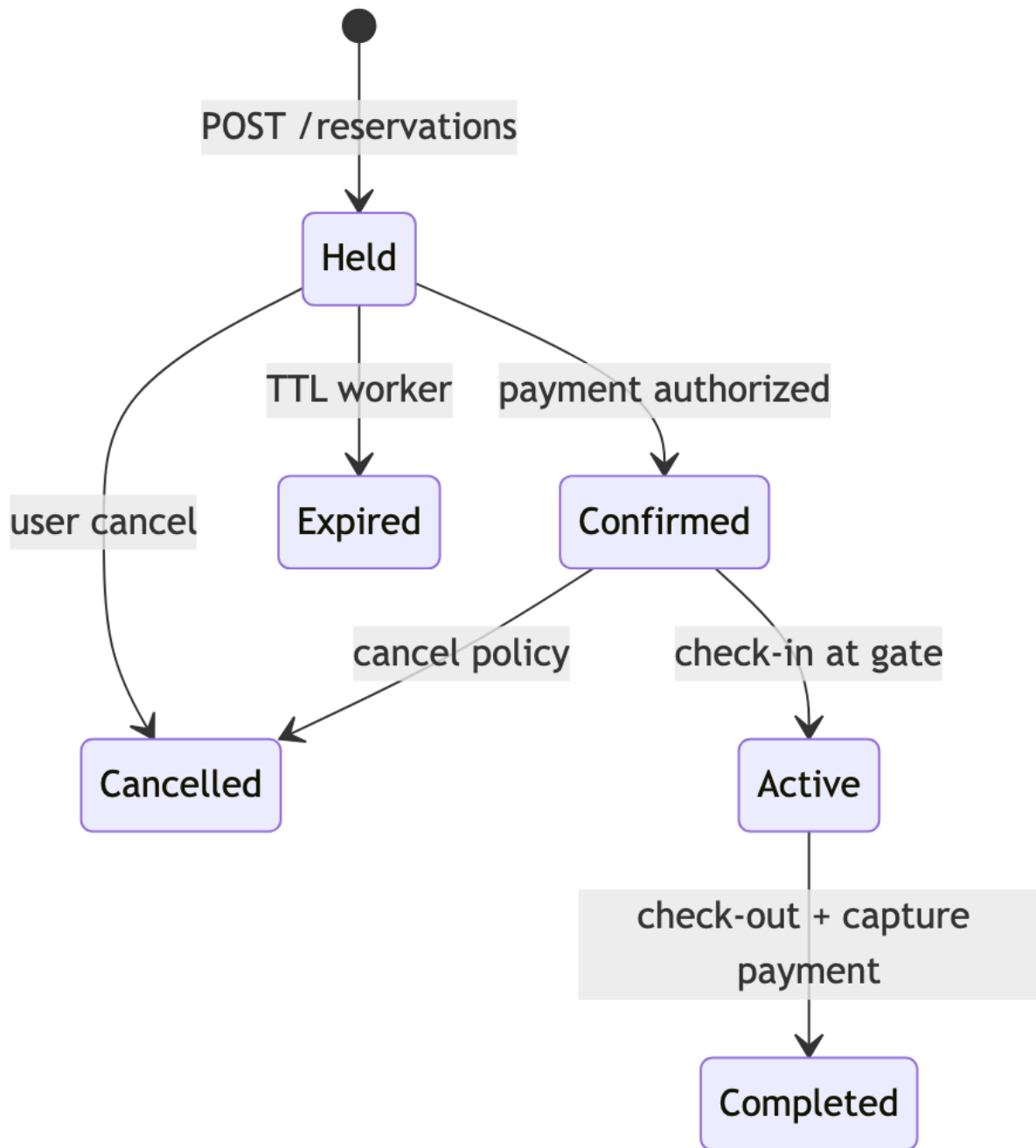


Figure 5: Diagram 5

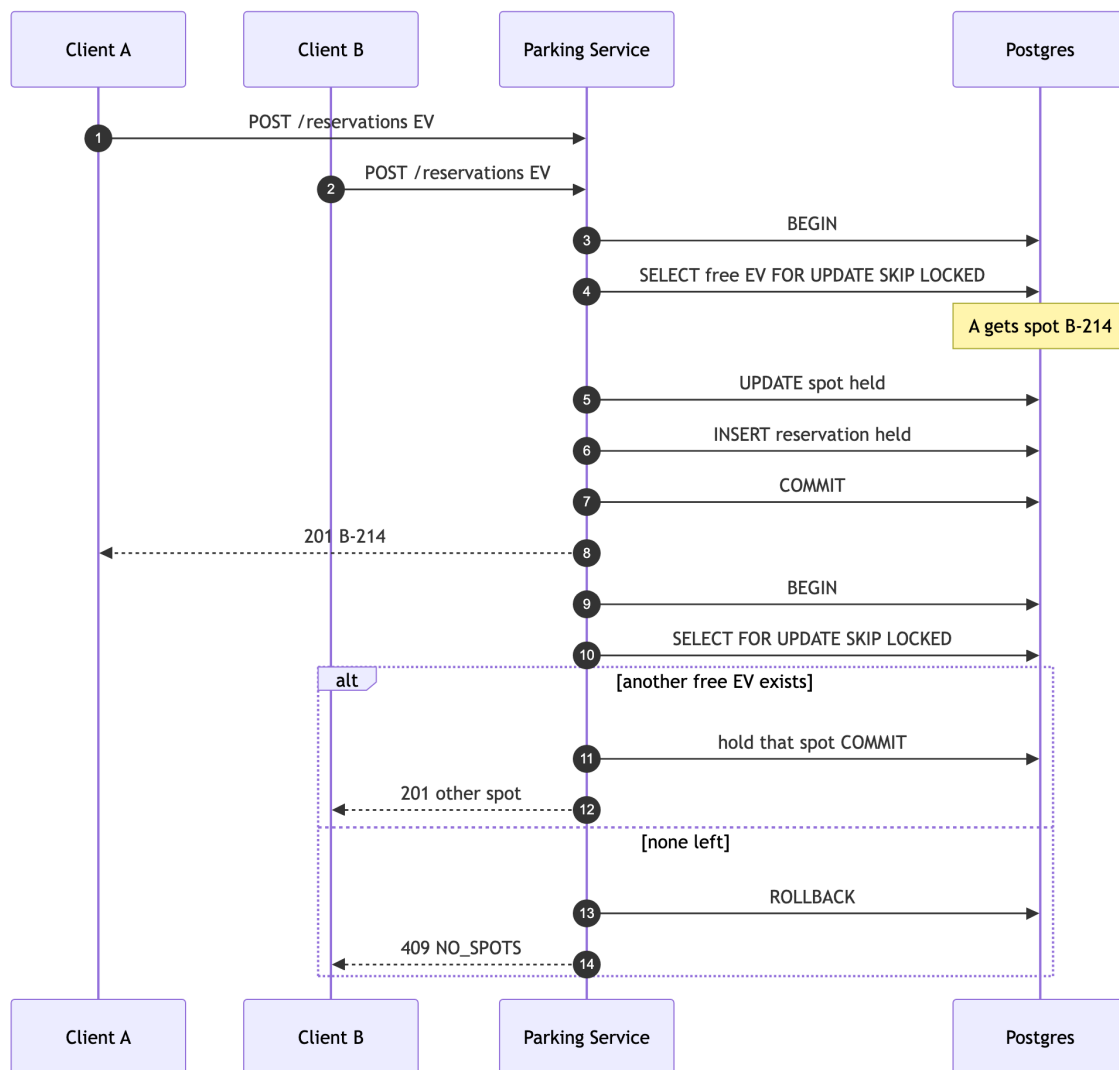


Figure 6: Diagram 6

1.12 Redis availability counters

```
INCR lot:{id}:free:ev    on free+held/occupied reverse
GET  lot:{id}:free:ev    for availability API
```

Rebuild from SQL on mismatch; cache is an optimization, not source of truth.

1.13 Pricing & payment failure

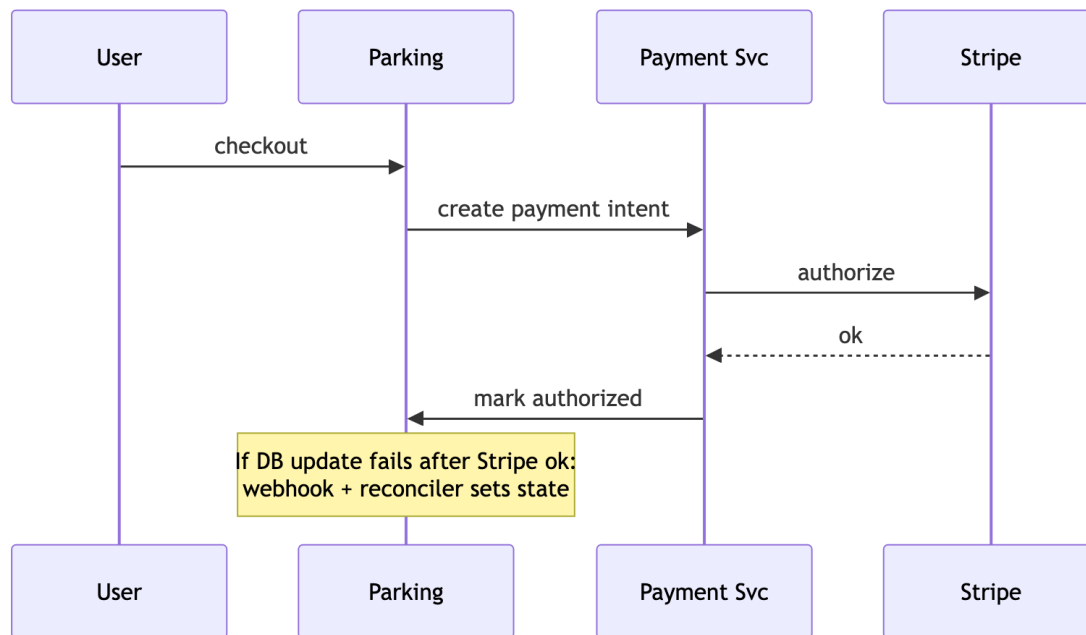


Figure 7: Diagram 7

1.14 Scale roadmap

Phase	Change
v1	Single Postgres, Redis counters, one lot
v2	Read replicas for analytics; partition reservations by month
v3	Shard by <code>lot_id</code> (each mega-lot its own DB); gate kiosks offline queue

1.15 Follow-up questions & answers

Interviewer	You
How do you prevent double assign?	Row lock / CAS on spot status in one transaction with insert.
Availability wrong?	Redis can drift; SQL is truth; periodic reconcile.

Interviewer	You
EV charger also scarce?	Model charger as second resource or spot subtype with own lock.
Overstay?	Session exit fee job; status stays occupied until paid.

1.16 Closing script (30 sec)

“Source of truth is Postgres with **FOR UPDATE SKIP LOCKED** on free spots. Holds expire via worker. Redis only accelerates availability reads. Payments are authorized on confirm and reconciled by webhook if we crash mid-flight.”

2. Car Rental (Enterprise) (C3 actual) — Full Playbook

2.1 Restatement

“Customers search vehicles by location/class/dates, hold one, pay, pick up and return. Invariant: no overlapping confirmed rentals for the same vehicle.”

2.2 Clarify

- One-way returns? Cross-branch?
- Insurance / add-ons / young-driver fees?
- Instant book vs request-to-book?
- Fleet size (10k cars)? Booking lead time?
- Corporate accounts / multi-driver?

2.3 NFRs

NFR	Target
Search p95	< 300 ms
Create booking p95	< 400 ms
Double-book risk	Zero for confirmed
Hold TTL	15 min
Peak search	2k QPS
Peak book	100 QPS

2.4 Architecture

Why split Search vs Booking? Different scaling and consistency: search may be slightly stale; booking must be correct.

2.5 ERD

2.6 Overlap invariant (critical diagram)

Postgres exclusion (strong answer):

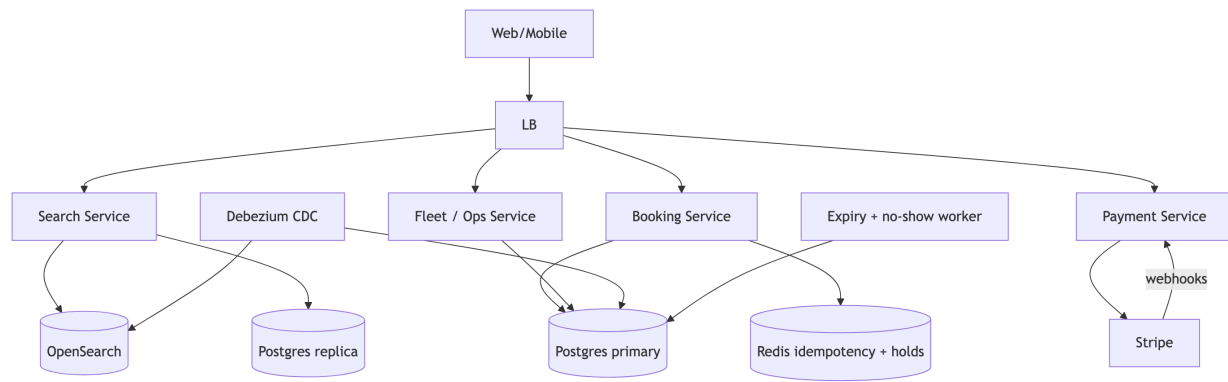


Figure 8: Diagram 8

```

ALTER TABLE booking ADD CONSTRAINT booking_no_overlap
EXCLUDE USING gist (
  vehicle_id WITH =,
  tstzrange(start_ts, end_ts, '[]') WITH &&
) WHERE (status IN ('held','confirmed','active'));

```

2.7 Booking sequence

2.8 APIs

```

GET /v1/vehicles/search?locationId&class&from&to&page
POST /v1/bookings
  { vehicleId, pickupLocationId, dropoffLocationId, from, to, addOns[], idempotencyKey }
POST /v1/bookings/{id}/confirm
POST /v1/bookings/{id}/cancel
POST /v1/bookings/{id}/extend { newEndTs }
POST /v1/bookings/{id}/pickup | /return
GET /v1/bookings/{id}

```

2.9 State machine

2.10 Scale & search freshness

- Indexer lag 1–5s acceptable if booking always re-validates in SQL.
- On book success: publish `vehicle_blocked` event to remove from search quickly.
- Shard bookings by `pickup_location_id` at very large scale.

2.11 Follow-ups

Q	A
Same car, different branches?	Track <code>location_id</code> ; one-way creates relocation job.
Maintenance window?	<code>MAINTENANCE</code> table participates in overlap checks.
Price change after hold?	Freeze rate snapshot on hold row.

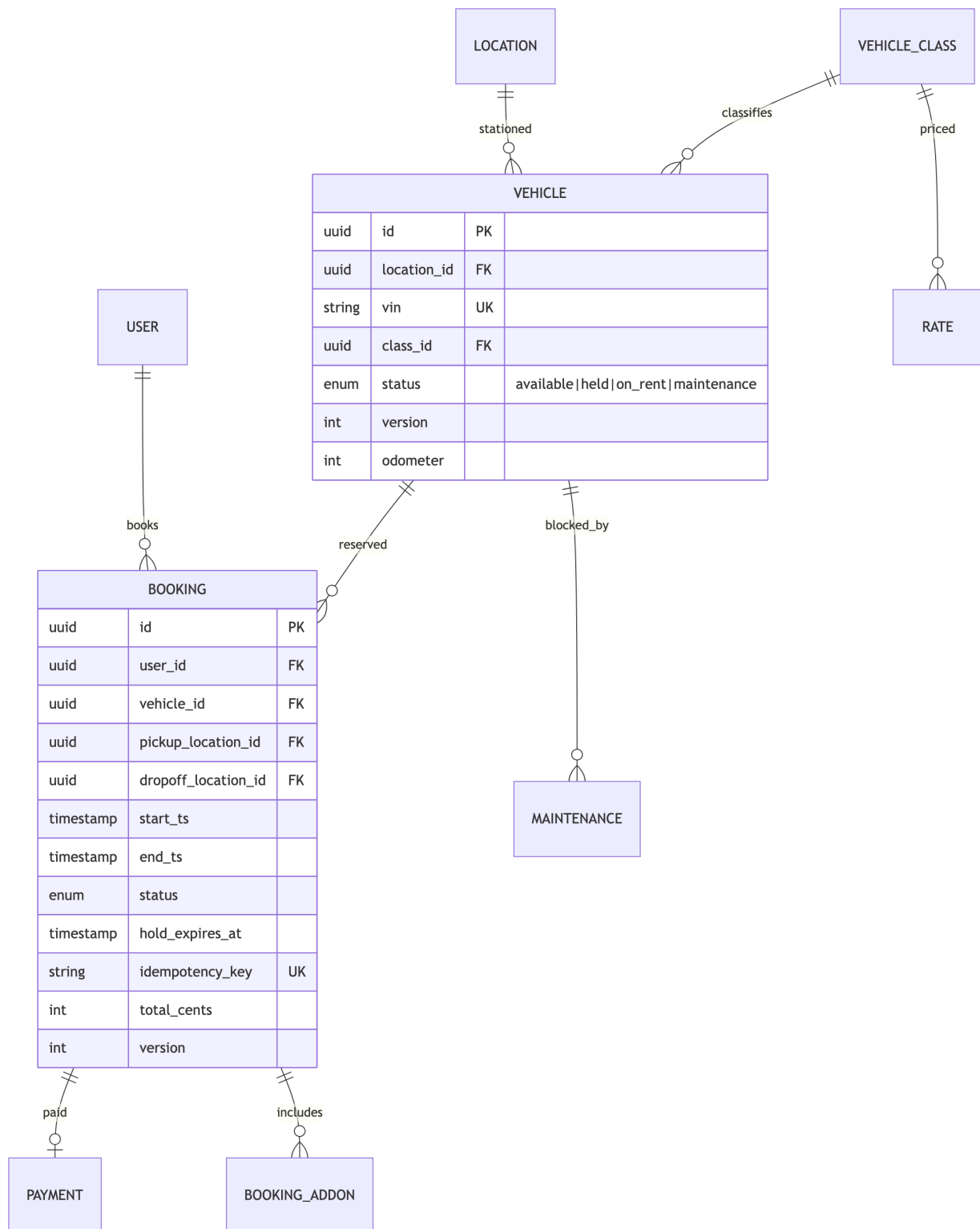


Figure 9: Diagram 9

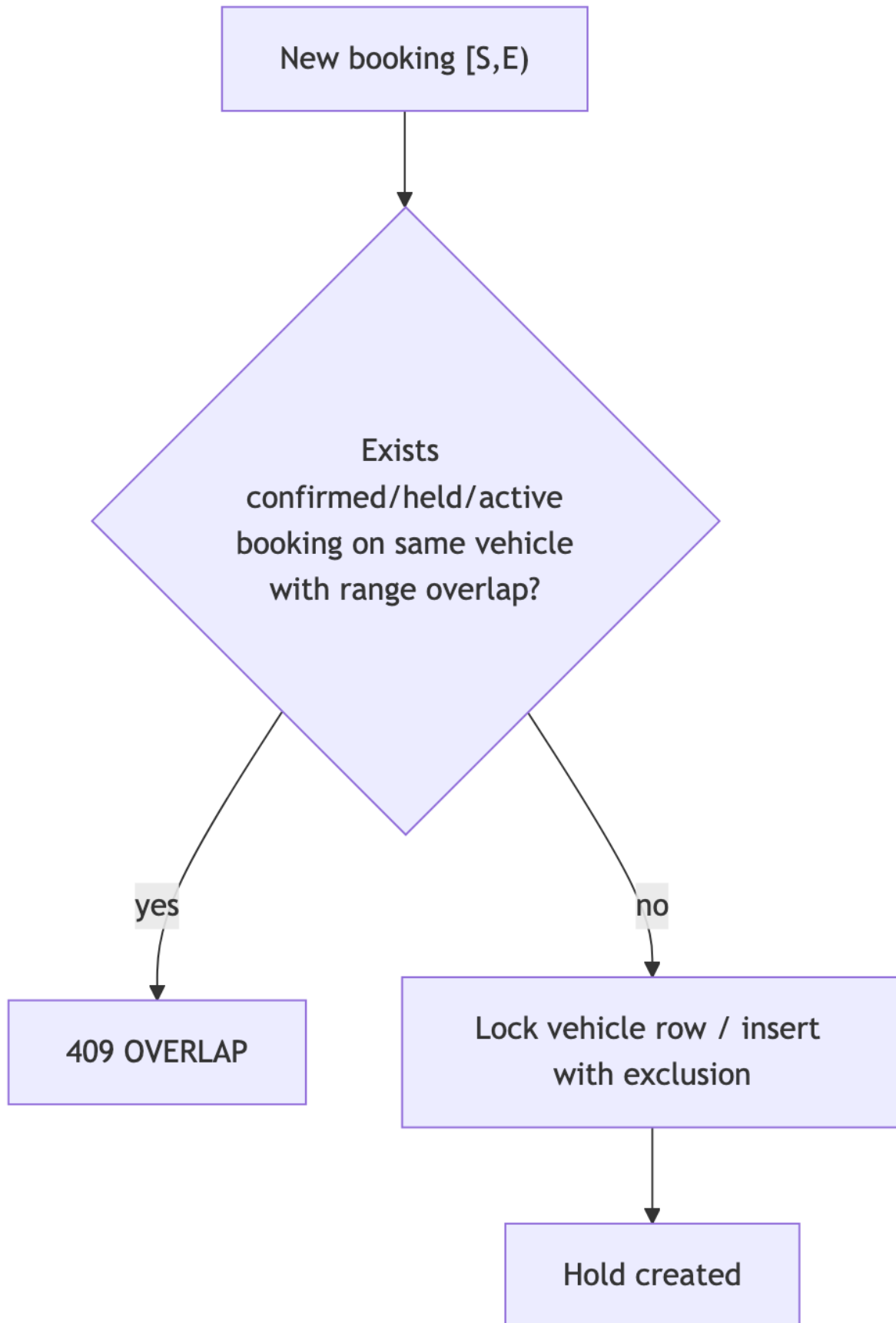


Figure 10: Diagram 10
15

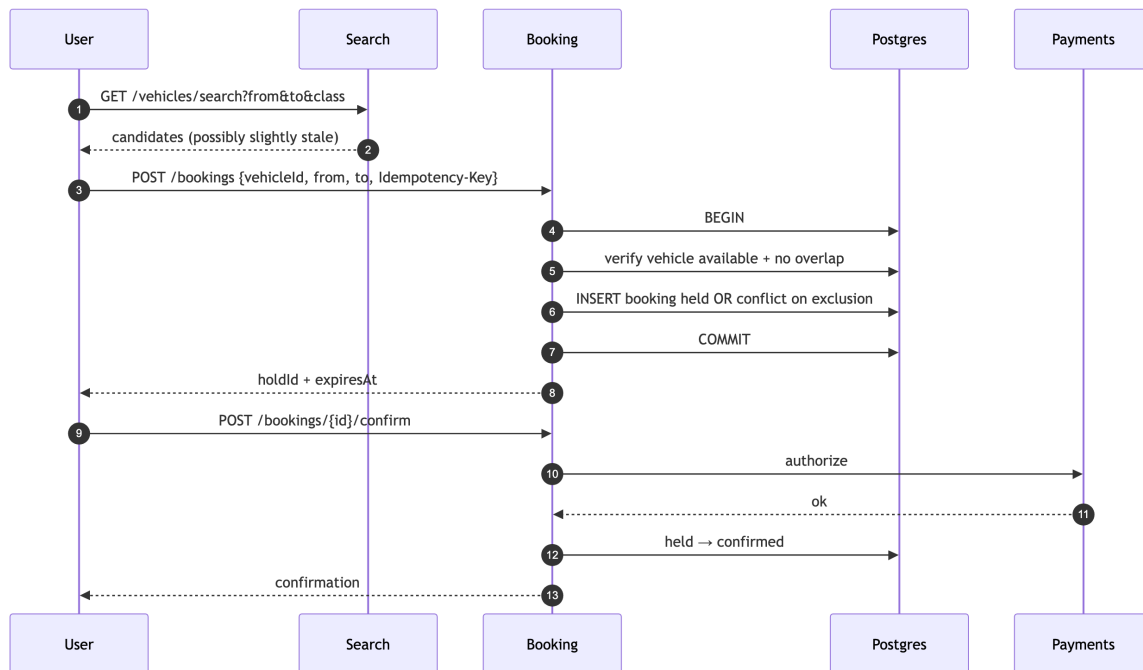


Figure 11: Diagram 11

2.12 Close

“Search is optimized for read scale and can be briefly stale. Booking re-checks and enforces a GiST exclusion on vehicle + time range so double rental is impossible. Holds TTL out; payments reconcile via webhooks.”

3. Metrics Logging & Aggregation (C3 actual) — Full Playbook

3.1 Restatement

“Services emit high-cardinality time series; we ingest reliably under spikes, store efficiently, and serve dashboard queries with rollups.”

3.2 Clarify

- Push agents vs pull Prometheus-style?
- Cardinality (unique series)? Histograms?
- Retention: raw 7d, 1m for 90d, 1h for 2y?
- Multi-tenant isolation? Alerting in scope?

3.3 Capacity sketch (say numbers)

```

20k hosts × 150 metrics × 1 sample/min
= 3e6 samples/min  50k samples/sec average
× 5 peak = 250k samples/sec
never write each sample as an OLTP row
  
```

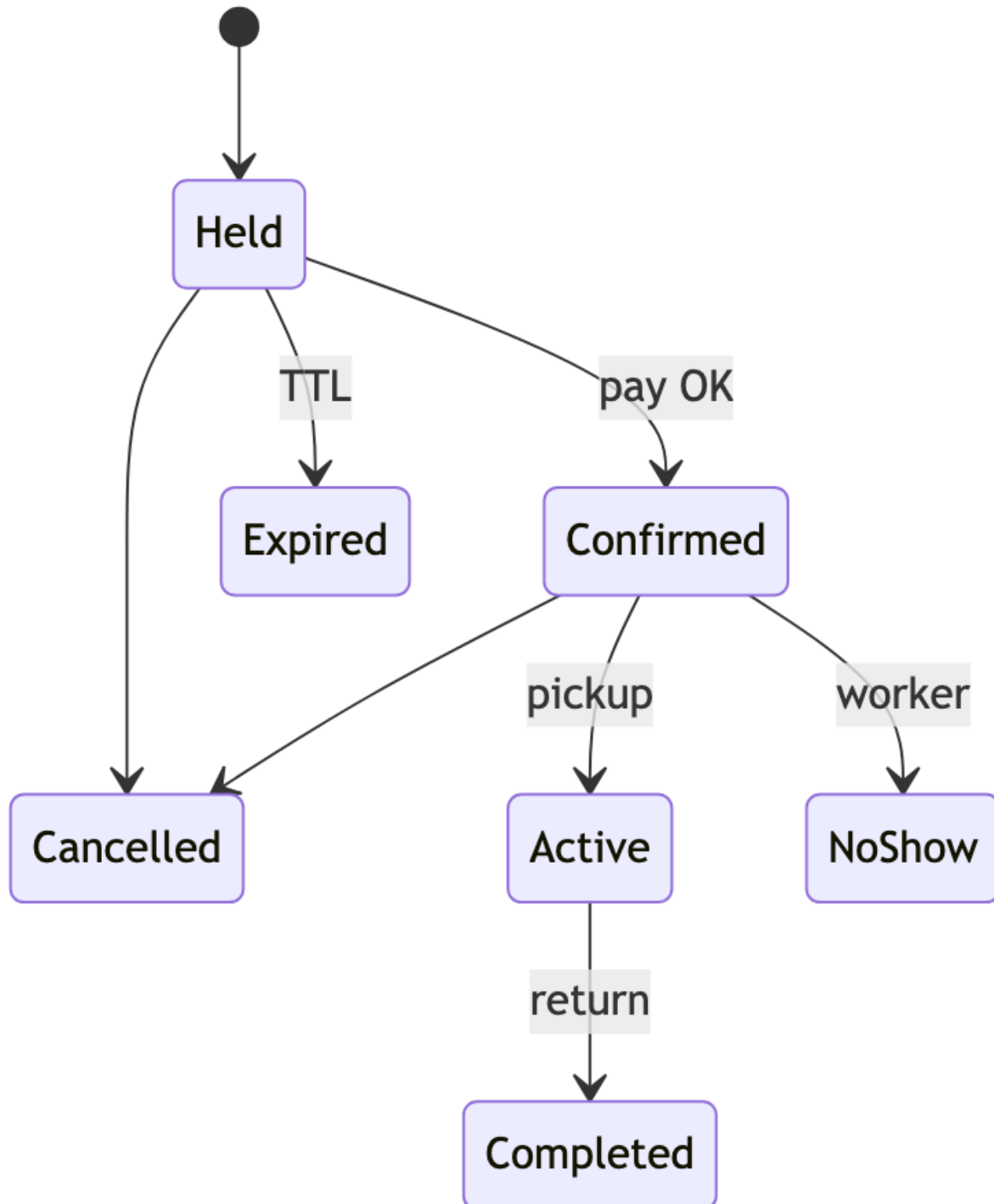



Figure 12: Diagram 12

3.4 Architecture (detailed)

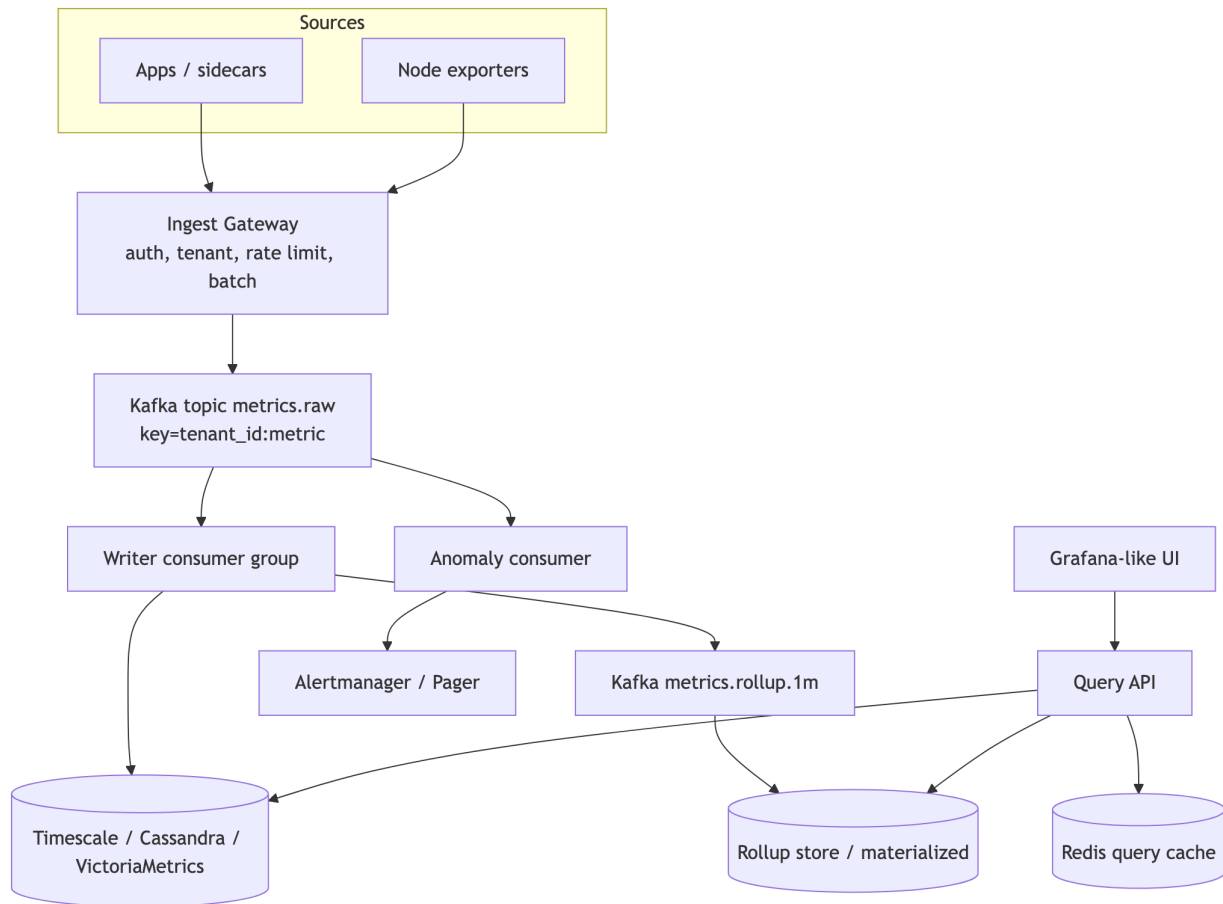


Figure 13: Diagram 13

3.5 Why each piece

Piece	Why
Gateway	Auth, tenant quotas, compression, 202 async accept
Kafka	Buffer spikes; replay; fan-out to writers + alertors
Partition key	Avoid hot partitions; keep tenant ordering if needed
TSDB	Wide/time-optimized writes; compression
Rollups	Dashboard queries must not scan raw forever
Cache	Repeated dashboard panels

3.6 Ingest sequence

3.7 Data model

```

Raw:      (tenant_id, metric, tags_hash, ts, value, type)
Rollup:   (tenant_id, metric, tags_hash, window_ts, count, sum, min, max, haves)
Series catalog: (tenant_id, metric, tags_json, created_at) -- control cardinality
  
```

Cardinality guard: reject or sample series above tenant budget.

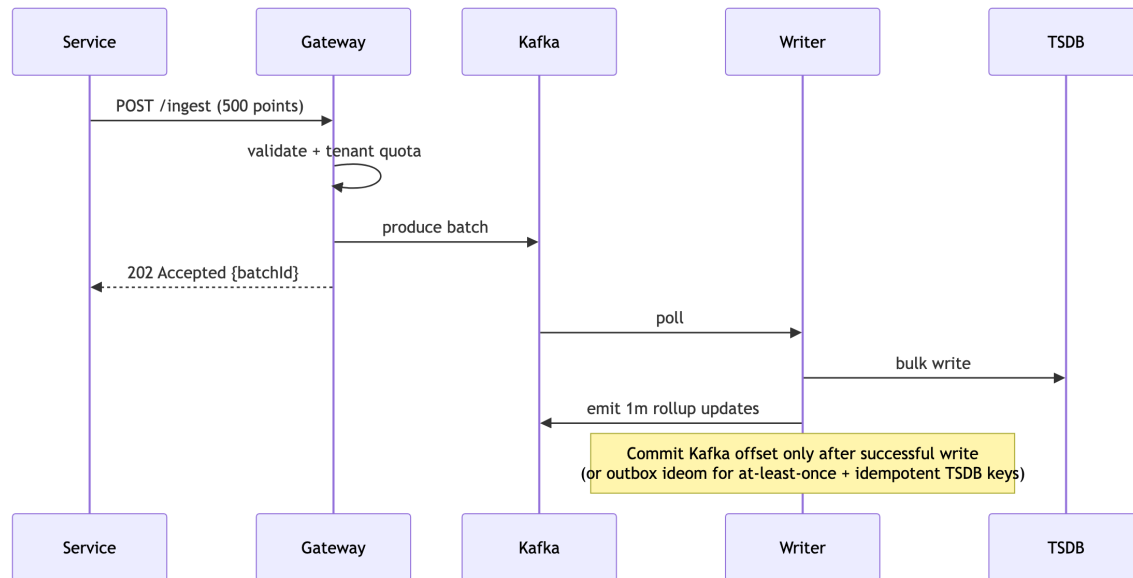


Figure 14: Diagram 14

3.8 Query API

```

POST /v1/query
{ "tenantId": "...", "metric": "http_req_latency_ms",
  "tags": { "route": "/checkout" }, "from": "...", "to": "...", "step": "1m",
  "agg": "p95" }
  
```

Query planner: if `step >= 1m` and `range > 2h` → rollup store; else raw.

3.9 Failure & consistency

- At-least-once from Kafka → idempotent writes with (`series_id`, `ts`) unique.
- Prefer **availability** on ingest; dashboards lag seconds.
- Poison batches → DLQ topic + alert.

3.10 Close

“Gateway + Kafka protect storage from bursts. TSDB holds raw hot data; rollups serve dashboards. We explicitly do not use relational inserts per point.”

4. Pastebin / Viral Text (C3 actual) — Full Playbook

4.1 Restatement

“Users publish text snippets and share a short link. Reads can go massively viral on a single key; writes are moderate.”

4.2 Clarify

Max size (1MB)? TTL? Password-private? Burn-after-read? Custom aliases? Auth required?

4.3 Capacity example

5M new pastes/day × 8 KB avg = 40 GB/day 1.2 TB/month content → object storage
 Reads 30× writes = 150M reads/day 1.7k QPS avg, 15-50k QPS peak on hot keys

4.4 Architecture

4.5 Write vs read paths

4.6 Schema

```
CREATE TABLE paste (
  id          BIGSERIAL PRIMARY KEY,
  short_code  TEXT UNIQUE NOT NULL,  -- base62
  user_id     UUID NULL,
  s3_key      TEXT NOT NULL,
  size_bytes  INT NOT NULL,
  visibility  TEXT NOT NULL,        -- public/unlisted/private
  password_hash TEXT NULL,
  expires_at  TIMESTAMPTZ NULL,
  created_at  TIMESTAMPTZ NOT NULL,
  view_count  BIGINT NOT NULL DEFAULT 0
);
```

4.7 Viral key protection

1. Redis + CDN for public pastes
2. Request coalescing (singleflight) on cache miss
3. Soft rate limit per IP on POST
4. View counts async (don't write Postgres on every GET)
5. Optional: replicate hot object to memory on multiple edges

4.8 Sequence (cache miss under load)

4.9 Close

“Split metadata and blob storage. The viral path is cache and CDN. The database never stores the body and never sees per-read writes.”

5. Elevator System (C3 OOD) — Full Playbook

5.1 Restatement

“Simulate elevators serving hall and cabin calls efficiently and safely. This is OOD + scheduling, not cloud architecture.”

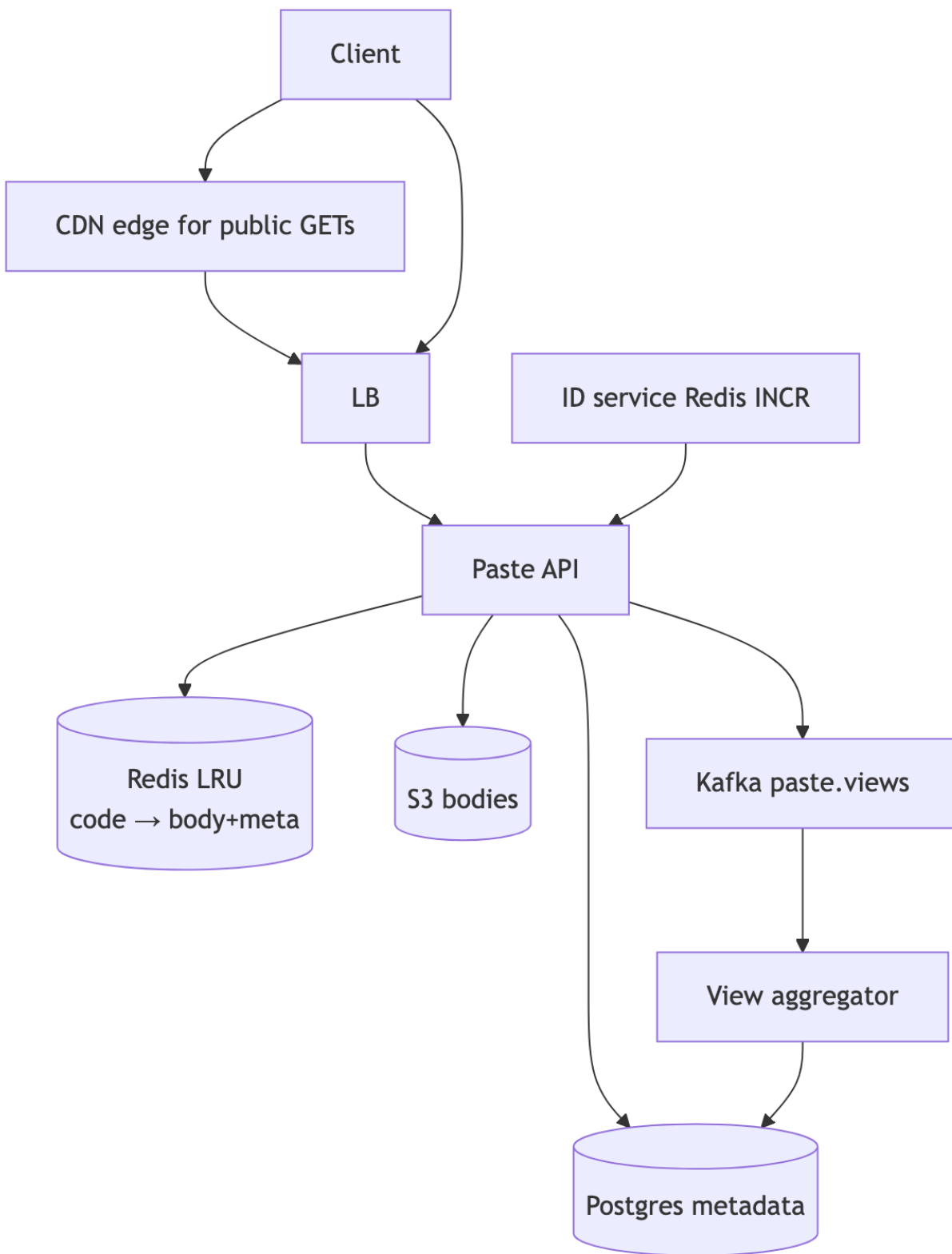


Figure 15: Diagram 15

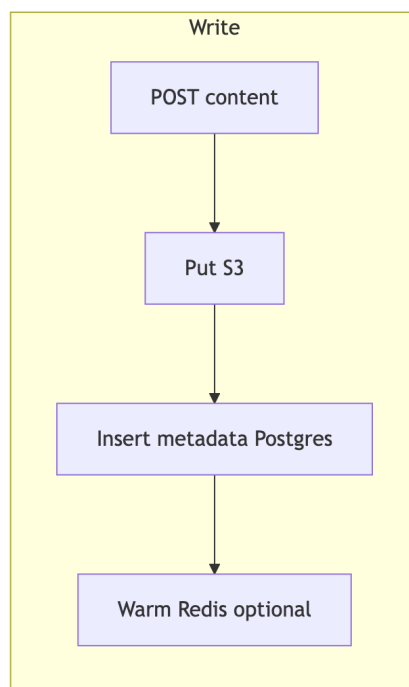
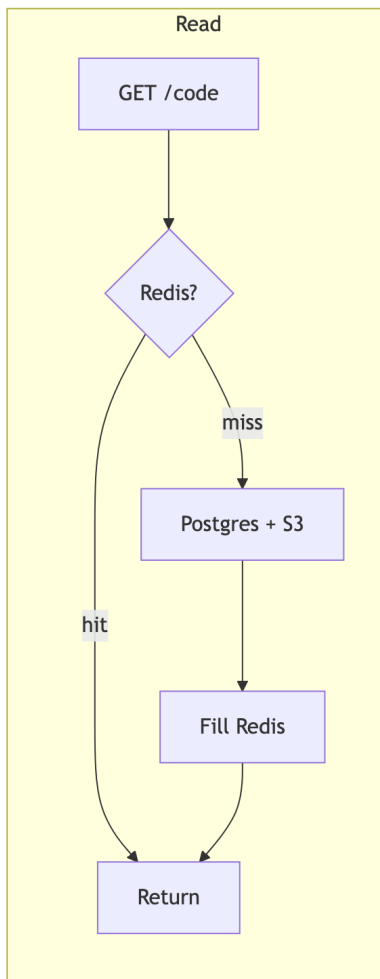


Figure 16: Diagram 16
22

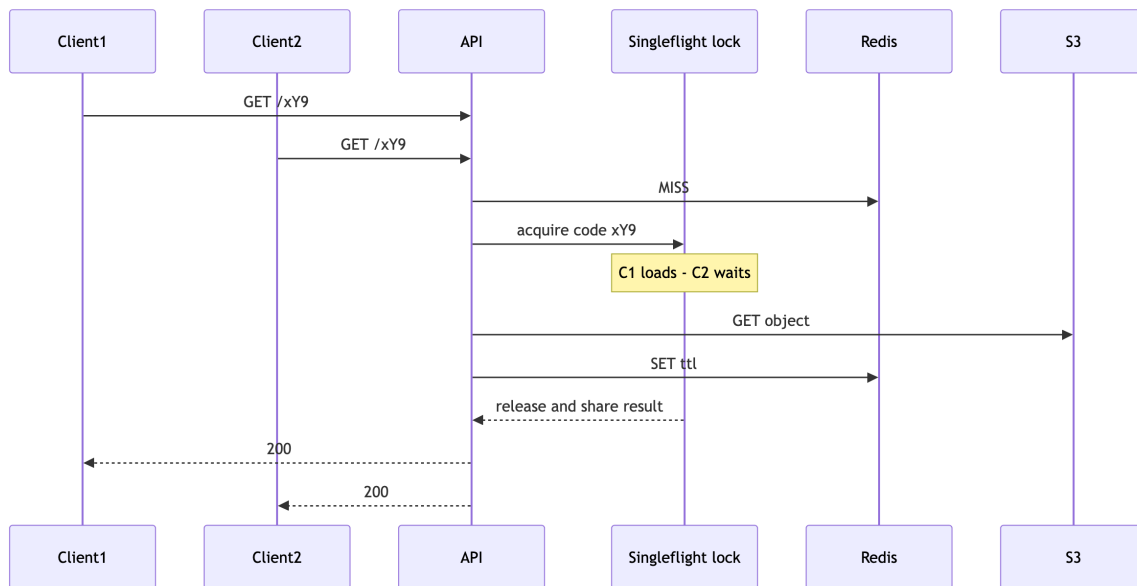


Figure 17: Diagram 17

5.2 Clarify

Floors? Elevator count? Max capacity? Peak-mode (up-peak morning)? Express elevators? Door timing?

5.3 Class diagram (detailed)

5.4 LOOK scheduling

5.5 Per-tick behavior

5.6 Threading model

- **Single-threaded simulator:** `controller.tick()` advances all elevators — easiest in interview.
- **Actor model:** each elevator mailbox — mention as production variant.

5.7 APIs / methods

```

POST hallCall(floor, direction)
POST cabinCall(elevatorId, floor)
GET status() -> [{id, floor, direction, state, stops}]
tick() // for simulation
  
```

5.8 Follow-ups

Q	A
Starvation?	Aging: increase priority of long-waiting hall calls.
Crowded car?	Reject cabin calls when at capacity; skip hall assign if load high.

Q	A
Emergency?	Clear stops; go to designated floor; state=MAINTENANCE.

5.9 Close

“Classes: Building, Controller, Elevator, Call. LOOK assignment by cost. Tick-based simulation keeps reasoning simple.”

6. Ticket / Event Booking (Ticketmaster pattern)

6.1 Architecture

6.2 Seat state machine

6.3 Deep dive

Hold seats with `SELECT ... FOR UPDATE` (or CAS status) for 10 minutes → pay → mark sold. Same scarce-resource pattern as parking/rental; cite this when interviewer asks about races.

6.4 Hot-event scale

- Shard inventory by section
- Queue waiting room (virtual queue) before hold API
- Idempotent holds per user+event

7. Ride-Sharing

7.1 Architecture

7.2 Matching sequence (detailed)

7.3 Schema highlights

```
Trip(id, rider_id, driver_id, status, pickup_geopoint, dropoff_geopoint,
      requested_at, assigned_at, fare_cents, version)
DriverStatus in Redis: {available|offered|busy|offline}, last_location, geohash
```

7.4 Hard problems to mention

- Offer stampede → limit concurrent offers; short offer TTL
- Ghost drivers → location freshness heartbeat
- Surge pricing → separate pricing service reading demand metrics
- Exact fare → finalize on trip end; authorize hold earlier

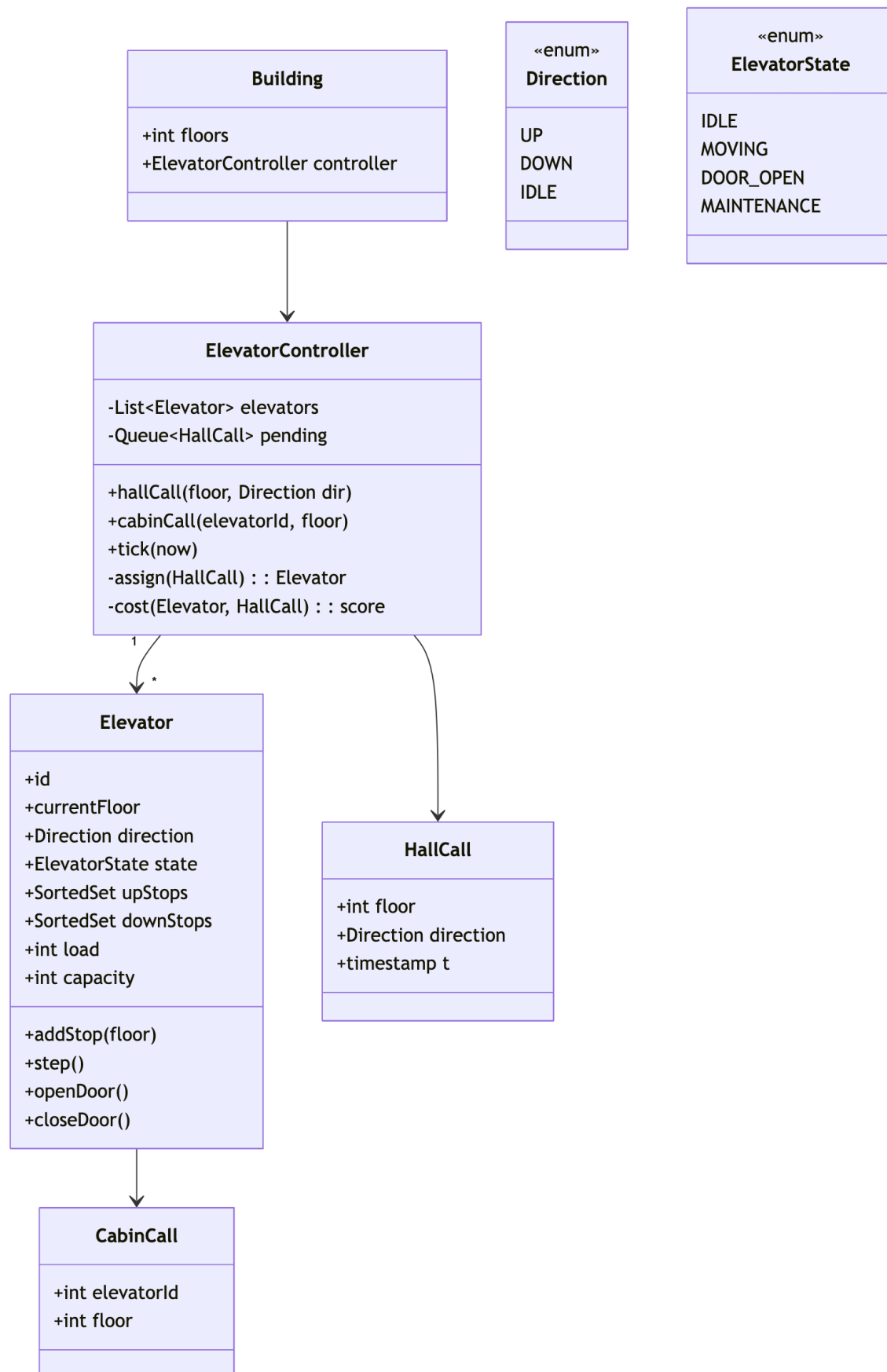


Figure 18: Diagram 18

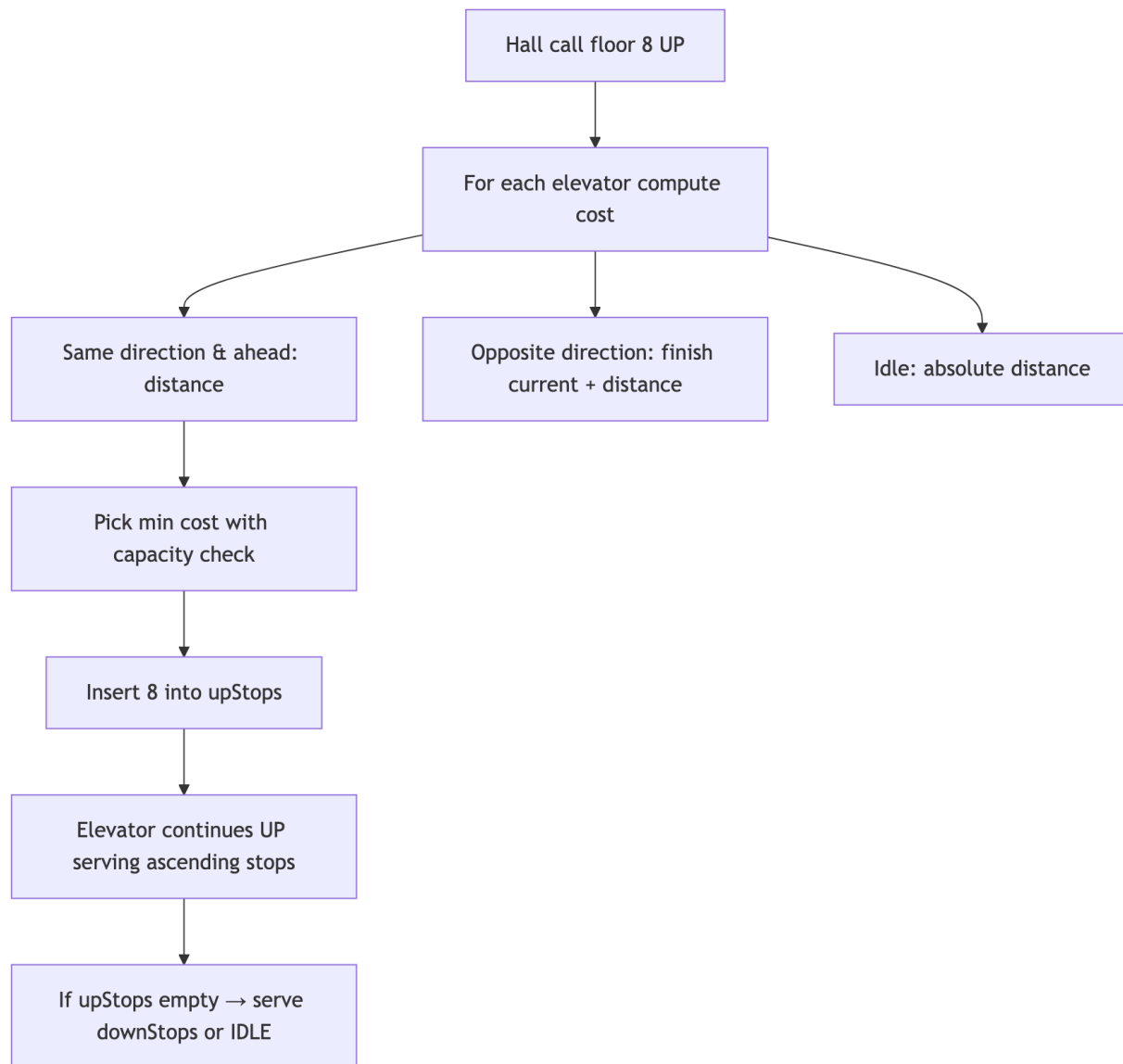


Figure 19: Diagram 19

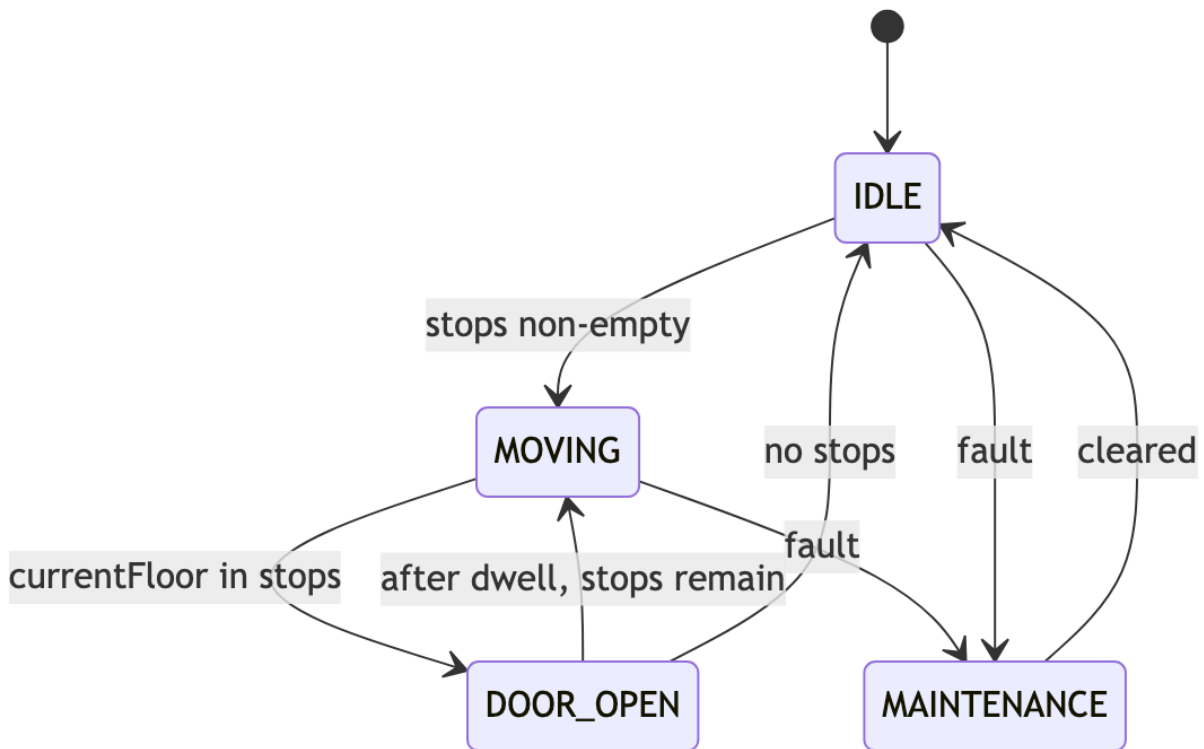


Figure 20: Diagram 20

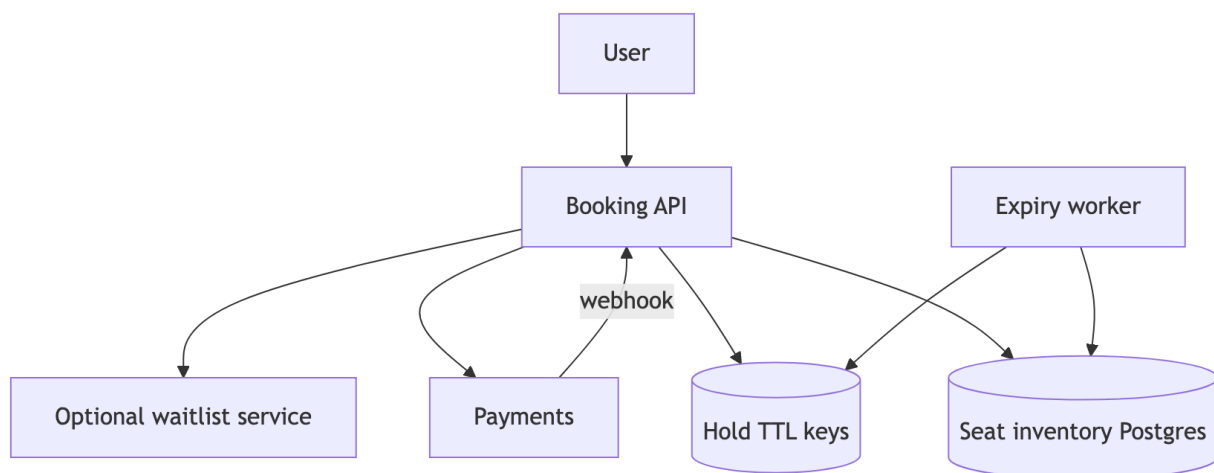


Figure 21: Diagram 21

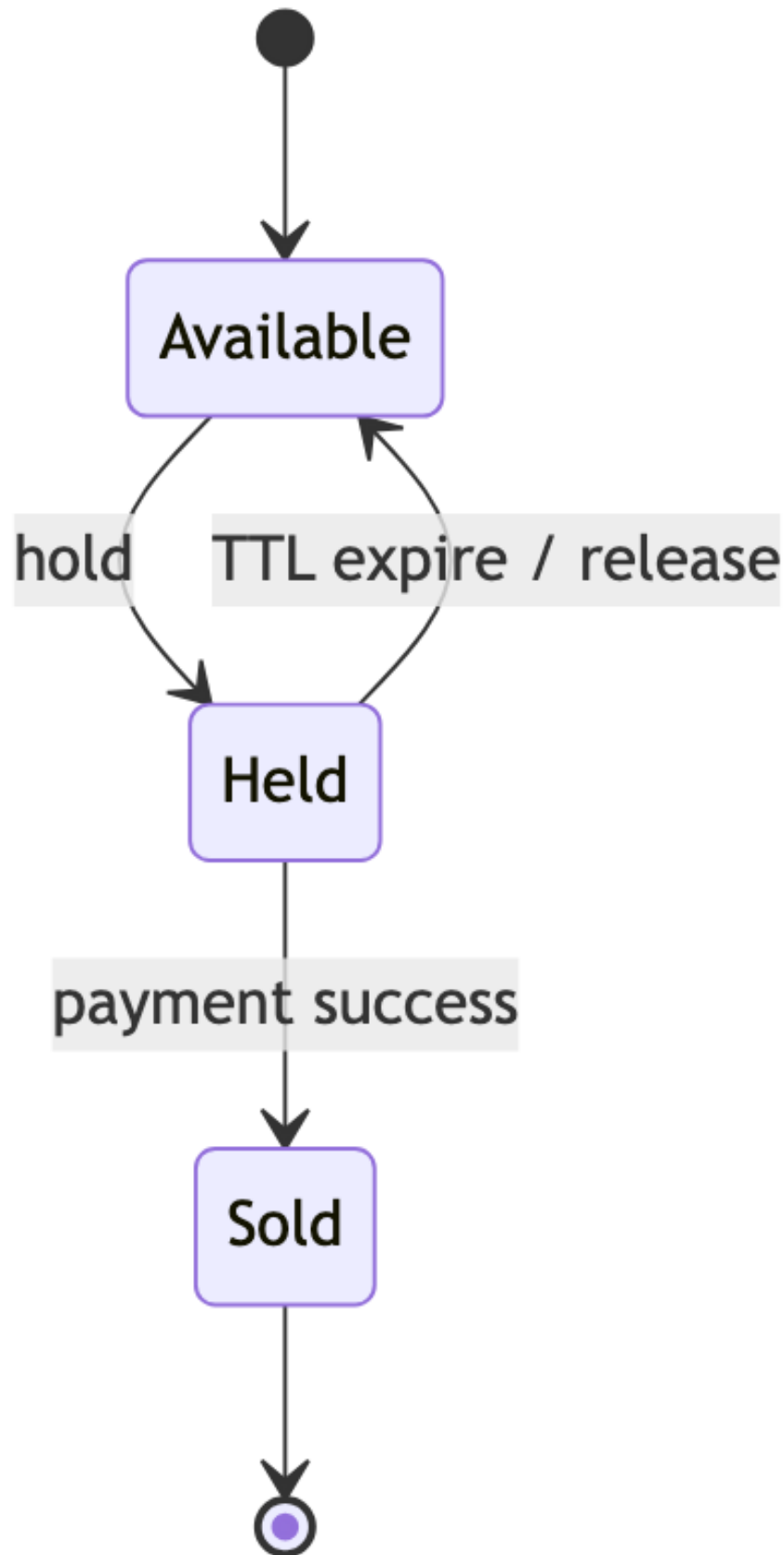


Figure 22: Diagram 22
28

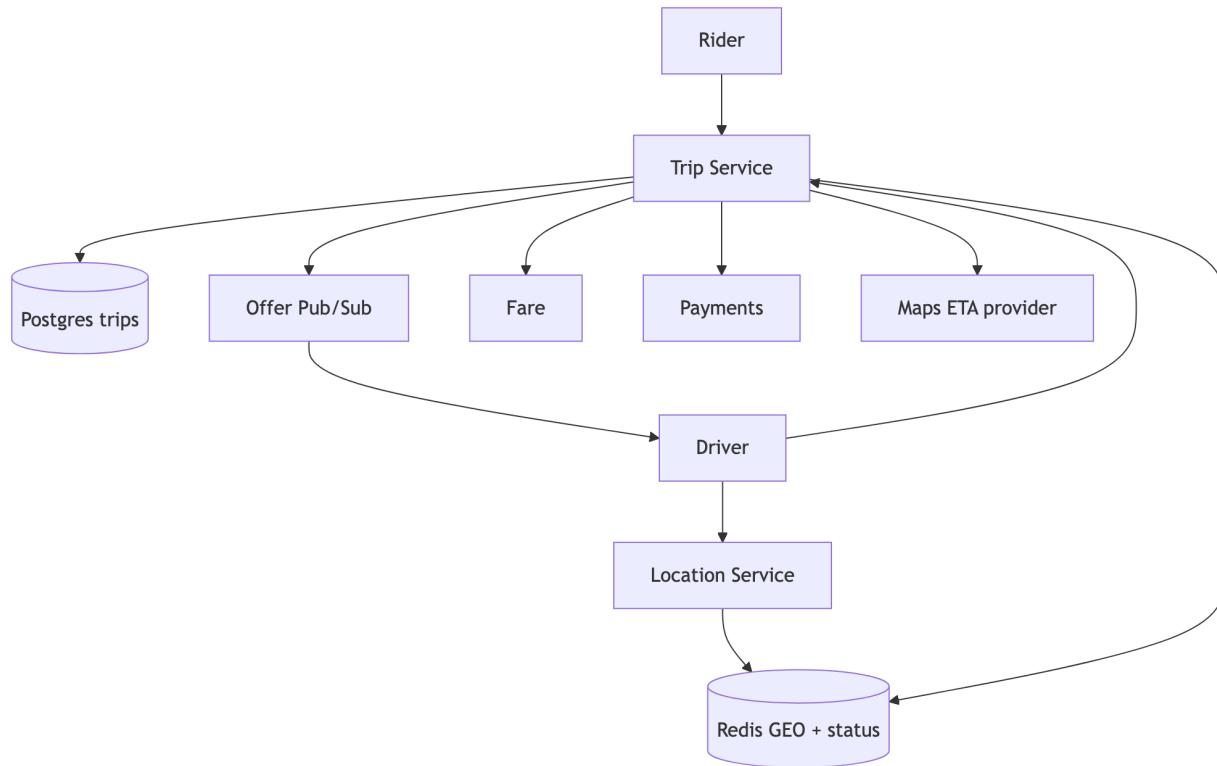


Figure 23: Diagram 23

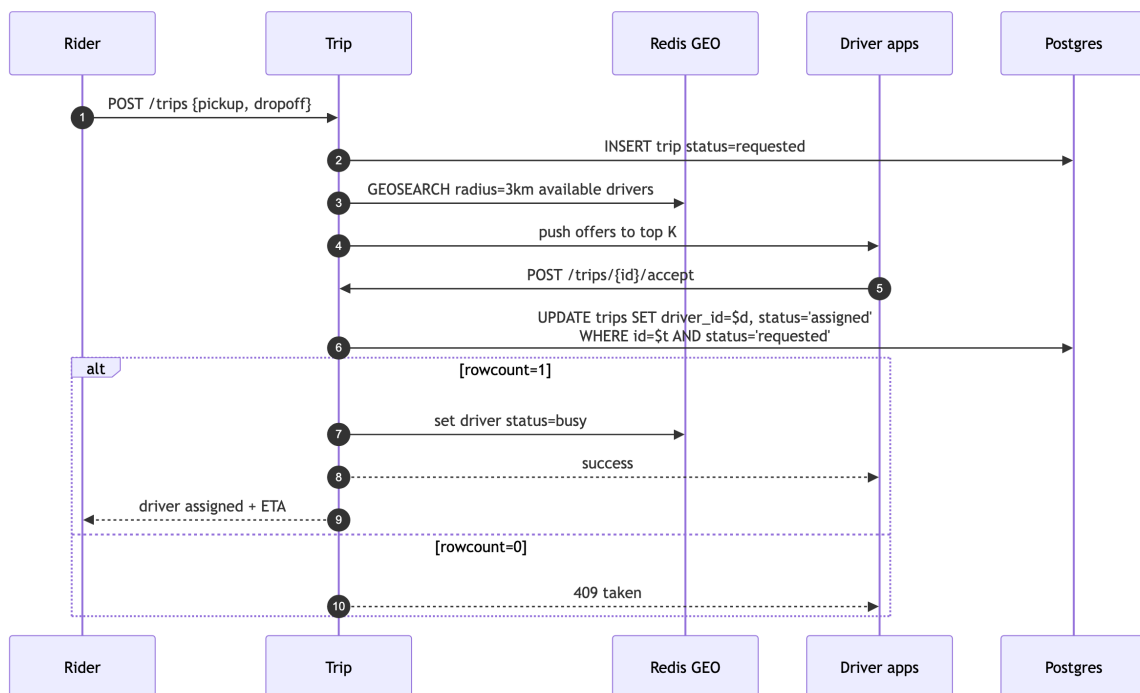


Figure 24: Diagram 24

8. Dropbox-like File Storage

8.1 Architecture

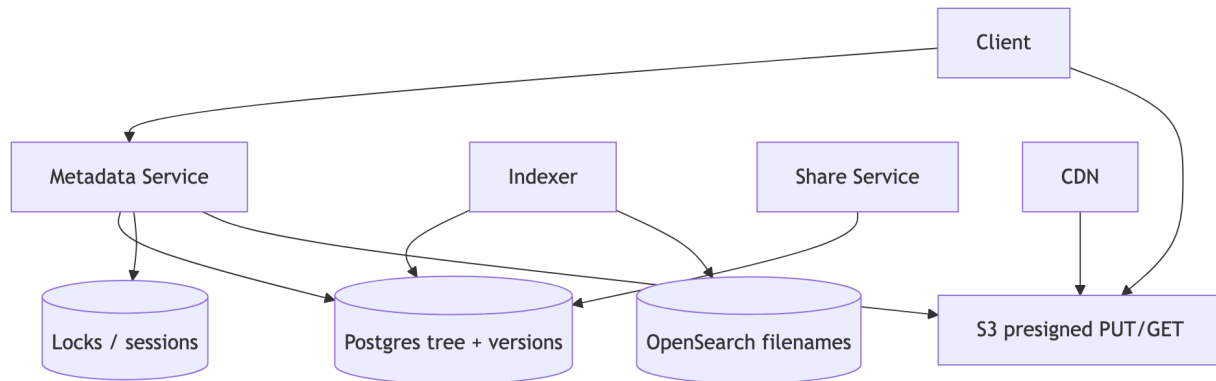


Figure 25: Diagram 25

8.2 Upload sequence

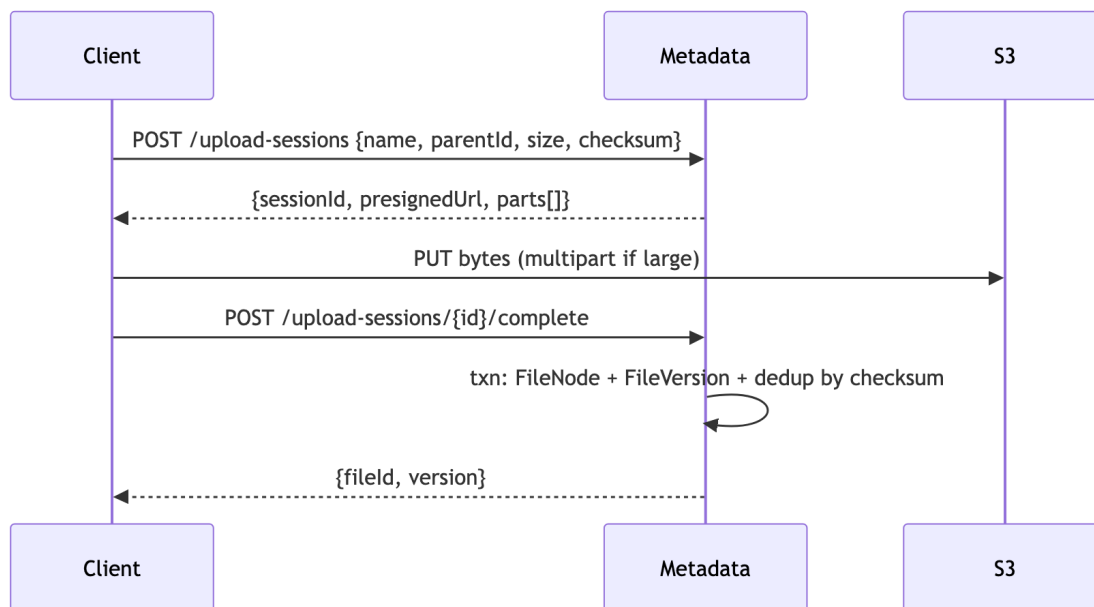


Figure 26: Diagram 26

8.3 Metadata schema

```

FileNode(id, parent_id, owner_id, name, type file|folder, is_deleted, unique(parent_id,name))
FileVersion(id, file_id, s3_key, size, checksum, version_num)
Share(id, node_id, grantee_id, permission read|write)
  
```

8.4 Hard topics

- Move/rename races → row lock on parent
- Sync conflicts → version vectors or LWW + conflict copy
- Dedup → content hash
- Large files → multipart + checksum per part

9. URL Shortener

9.1 Architecture

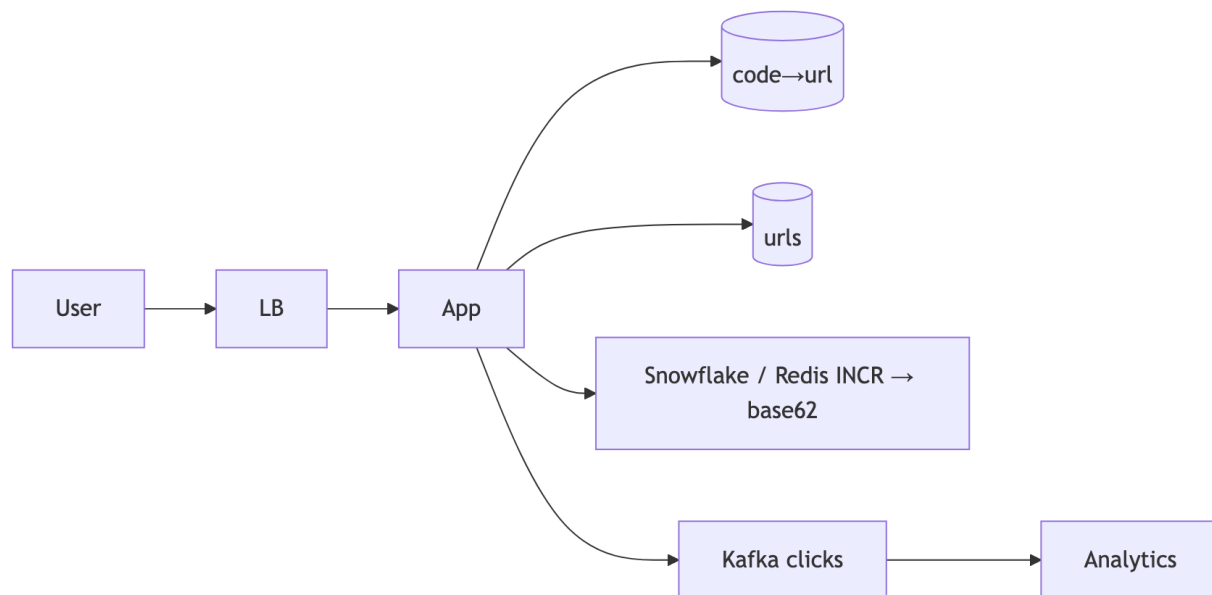


Figure 27: Diagram 27

9.2 Design choices table

Decision	Option A	Option B	Pick when
ID	Counter base62	Hash long URL	Counter: predictable size; Hash: dedup
Redirect	301	302	302 if counting clicks
Storage	SQL	KV	SQL fine to millions/day with cache

9.3 Redirect path

Cache-first; on miss load SQL; fill cache; async click event.

10. Rate Limiter / Hit Counter

10.1 Placement

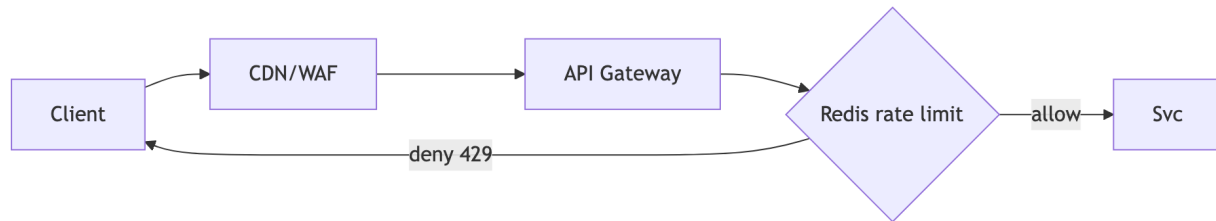


Figure 28: Diagram 28

10.2 Algorithms (detail)

Token bucket * Capacity C , refill r tokens/sec

* Atomic Redis Lua: refill based on clock, then take 1

Sliding window counter * Current minute + previous minute weighted by overlap

Hit counter (coding → design) * Single node: arrays size 300

* Multi node: Kafka → aggregator OR Redis INCR with TTL buckets

10.3 Redis Lua sketch (say verbally)

```

tokens = min(C, tokens + (now-last)*r)
if tokens < 1: deny else tokens--, allow
  
```

11. Enterprise RAG / Agent Platform

11.1 Architecture (detailed)

11.2 Grounded answer sequence

11.3 Tables

```

Tenant, Document(acl_roles[]), Chunk(doc_id, ordinal, text, hash),
EmbeddingRef(chunk_id, model, vector_id),
Conversation, Message, Citation(message_id, chunk_id, span),
ToolCall(status, args, result), AuditEvent
  
```

11.4 NFRs to emphasize (FDE / C3)

- ACL **before** retrieval (security > recall)
- Citations mandatory
- Private model endpoint / VPC

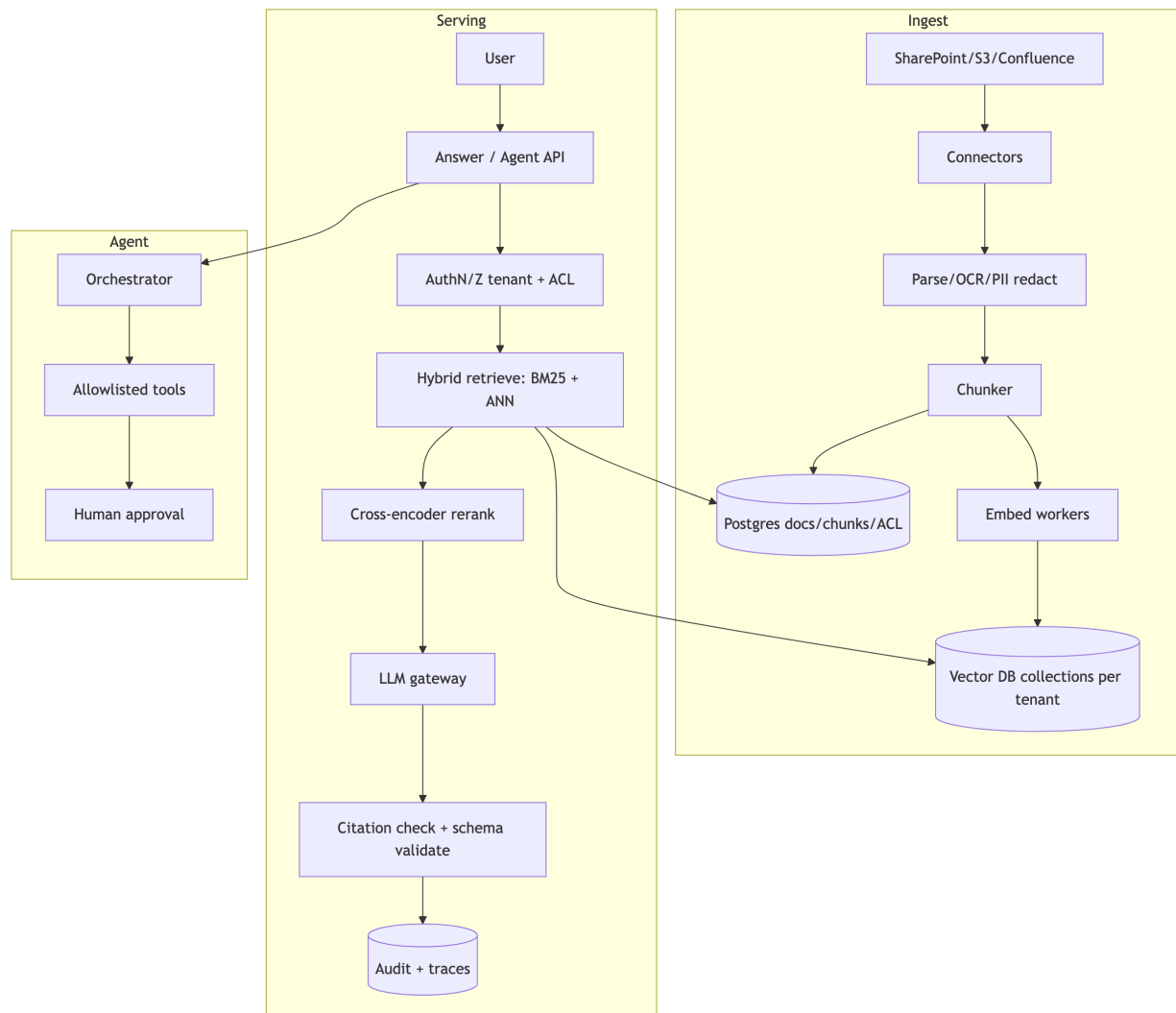


Figure 29: Diagram 29

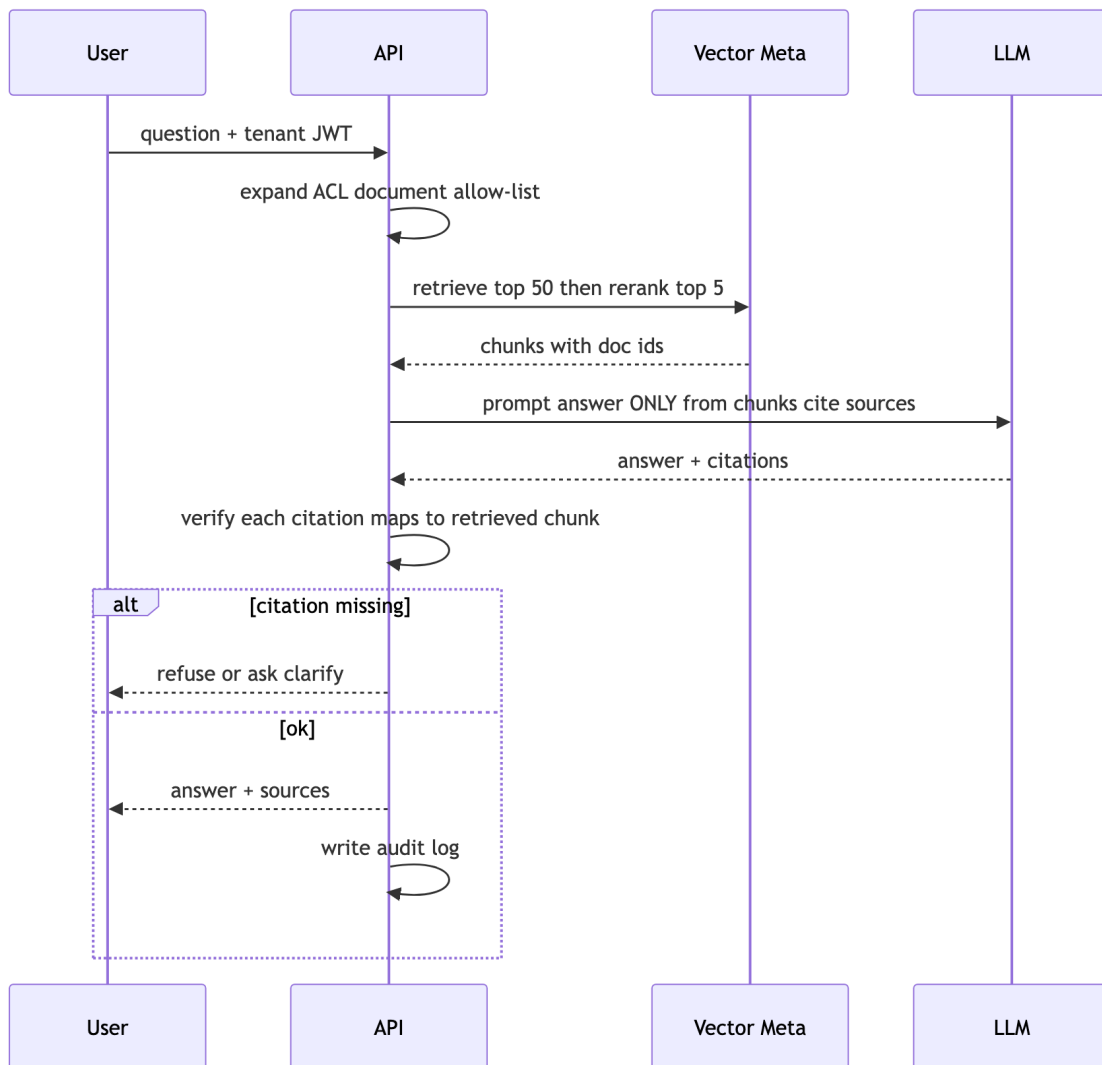


Figure 30: Diagram 30

- Eval set + prompt versioning before prod promote
- Cost controls: max tokens, cache embeddings

11.5 Close

“Ingest and query are separate pipelines. Retrieval is ACL-aware. The model is not the source of truth—chunks are. Agents get typed tools, budgets, and audit logs.”

12. IoT / Telemetry Ingestion

12.1 Architecture

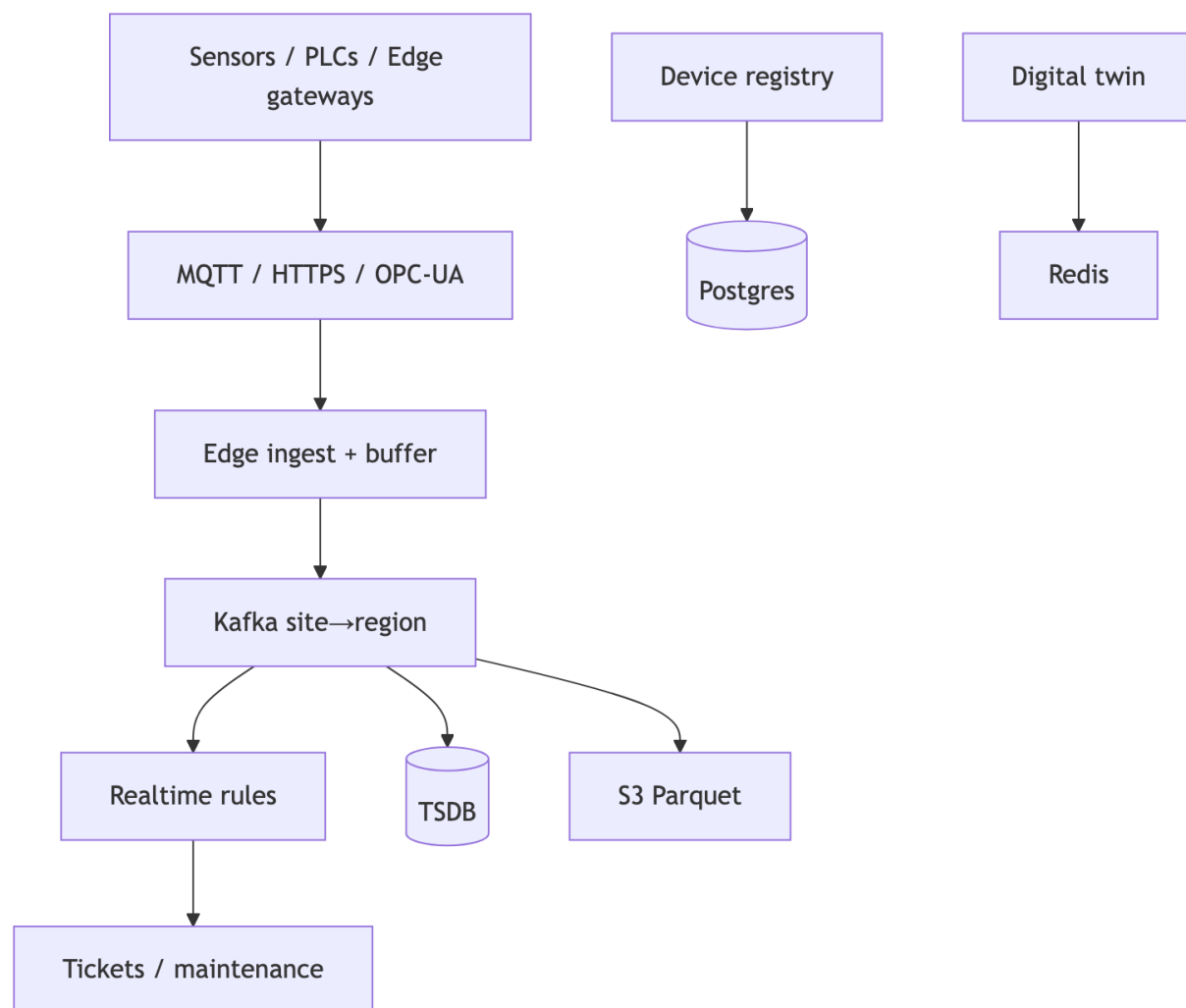


Figure 31: Diagram 31

12.2 Guarantees

Issue	Approach
Dupes	Idempotent key (device_id, event_id)
Clock skew	Store device_ts + received_ts
Backfill storm	Edge buffer + Kafka lag autoscaling
Exactly-once	Effectively-once via idempotent sinks

12.3 Device twin

Redis: device:{id} → {state, last_seen, firmware, config_version}
 Postgres: durable registry + credentials

13. Notification System

13.1 Architecture

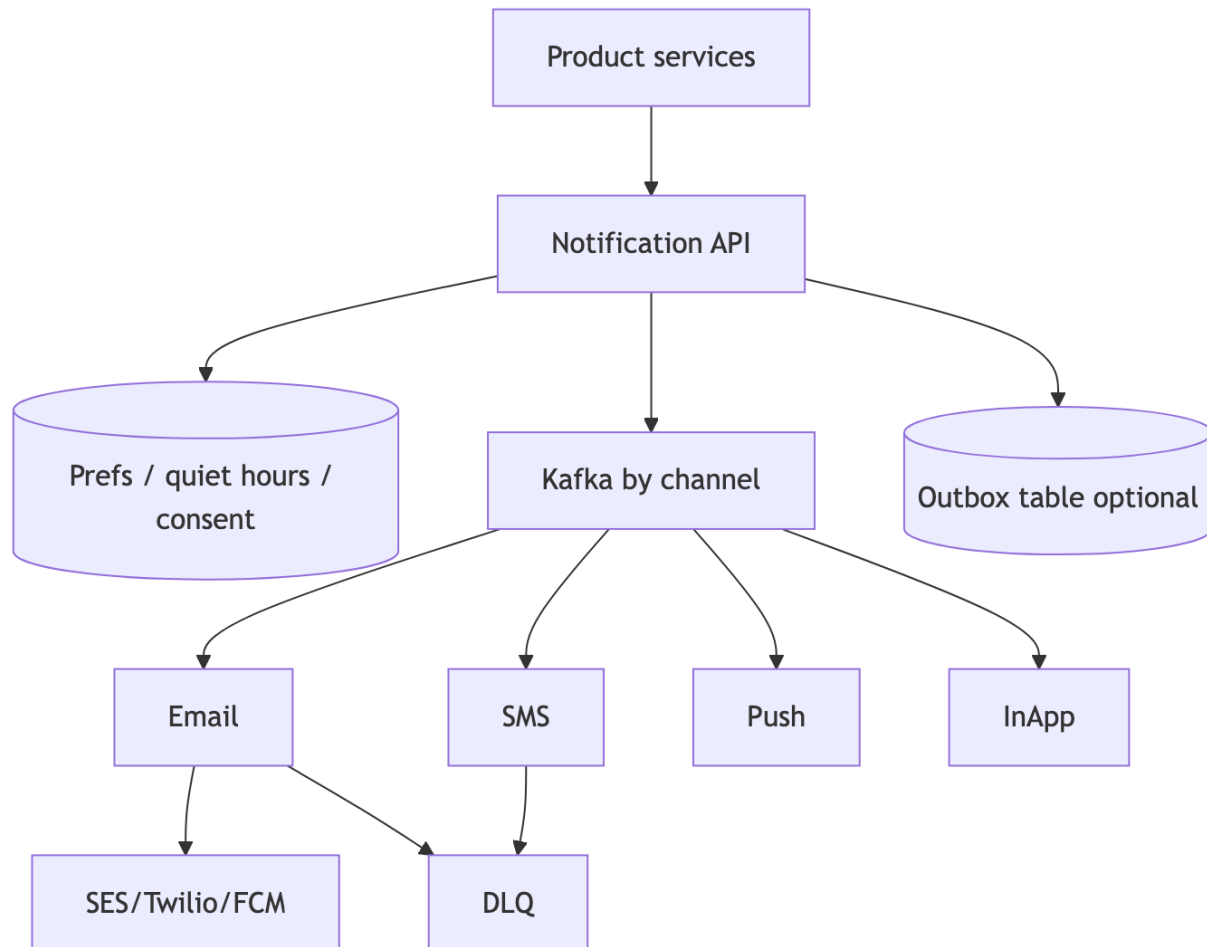


Figure 32: Diagram 32

13.2 Deduping

Unique (user_id, template_id, dedupe_key) for 24h. Retries use same key.

13.3 Fan-out

Large audience → chunk into 1k-user jobs; progress in DB; don't put 10M IDs in one message.

14. Chat / Messaging

14.1 Architecture

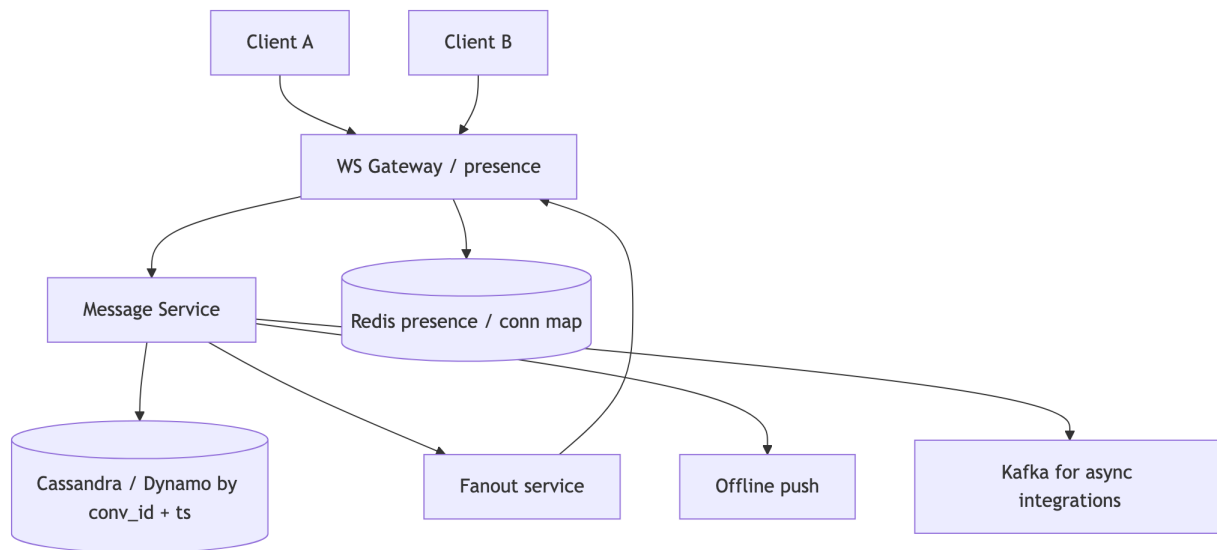


Figure 33: Diagram 33

14.2 Delivery guarantees

- Client client_msg_id for idempotent send
- Per-connection ACK; store-and-forward if offline
- Read receipts async

14.3 Ordering

Order within a conversation partition; don't claim global total order.

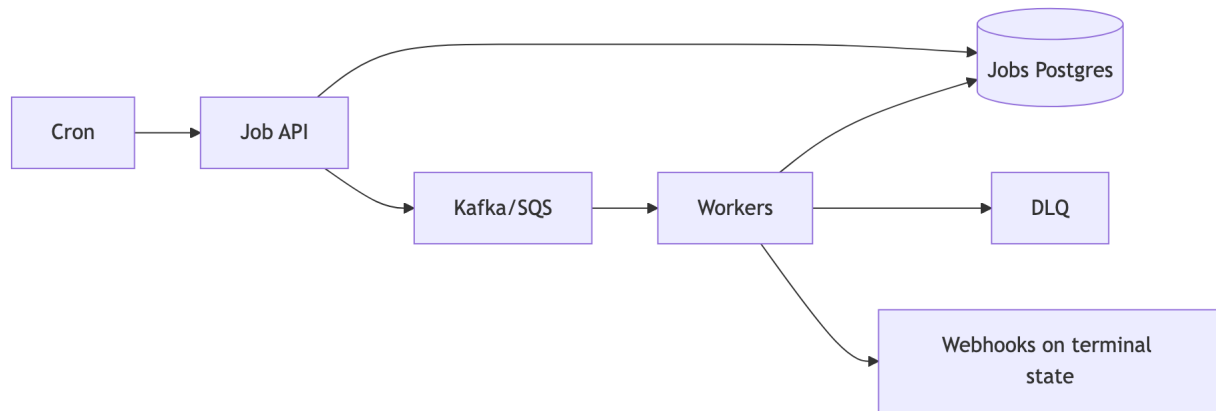


Figure 34: Diagram 34

15. Distributed Job / Workflow System

15.1 Architecture

15.2 State machine

15.3 Worker contract

1. Receive message
2. CAS `queued` → `running` (prevent double run)
3. Execute idempotently
4. Mark terminal + commit offset
5. On crash: visibility timeout returns to queue

16. Feature Store / Model Serving

16.1 Architecture

16.2 Talk track

Point-in-time joins for training; online path only uses features available at request time; registry supports rollback; shadow mode before promote.

17. Cross-Cutting Patterns

17.1 Universal booking pattern

17.2 Outbox pattern

17.3 Cache patterns

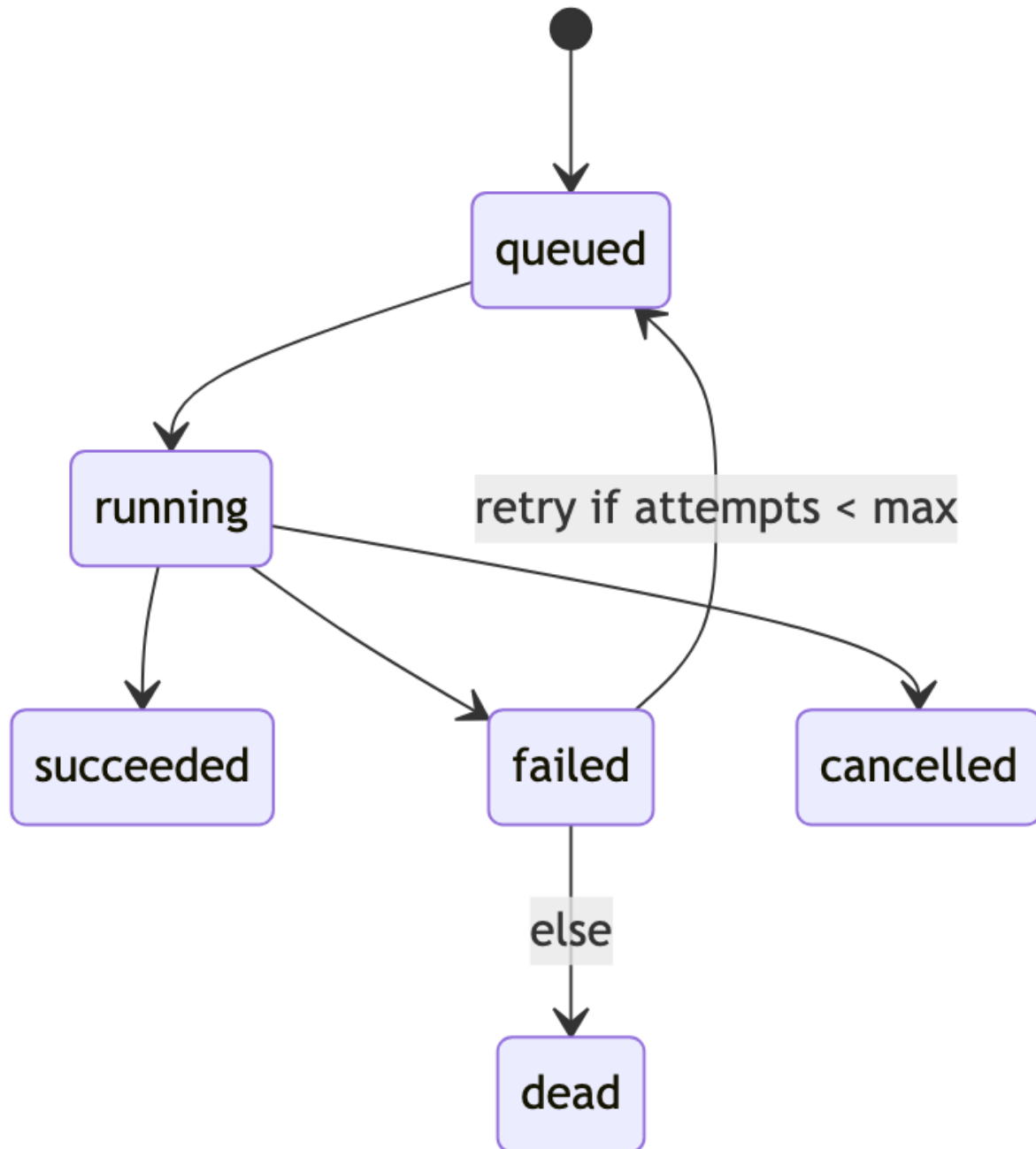


Figure 35: Diagram 35

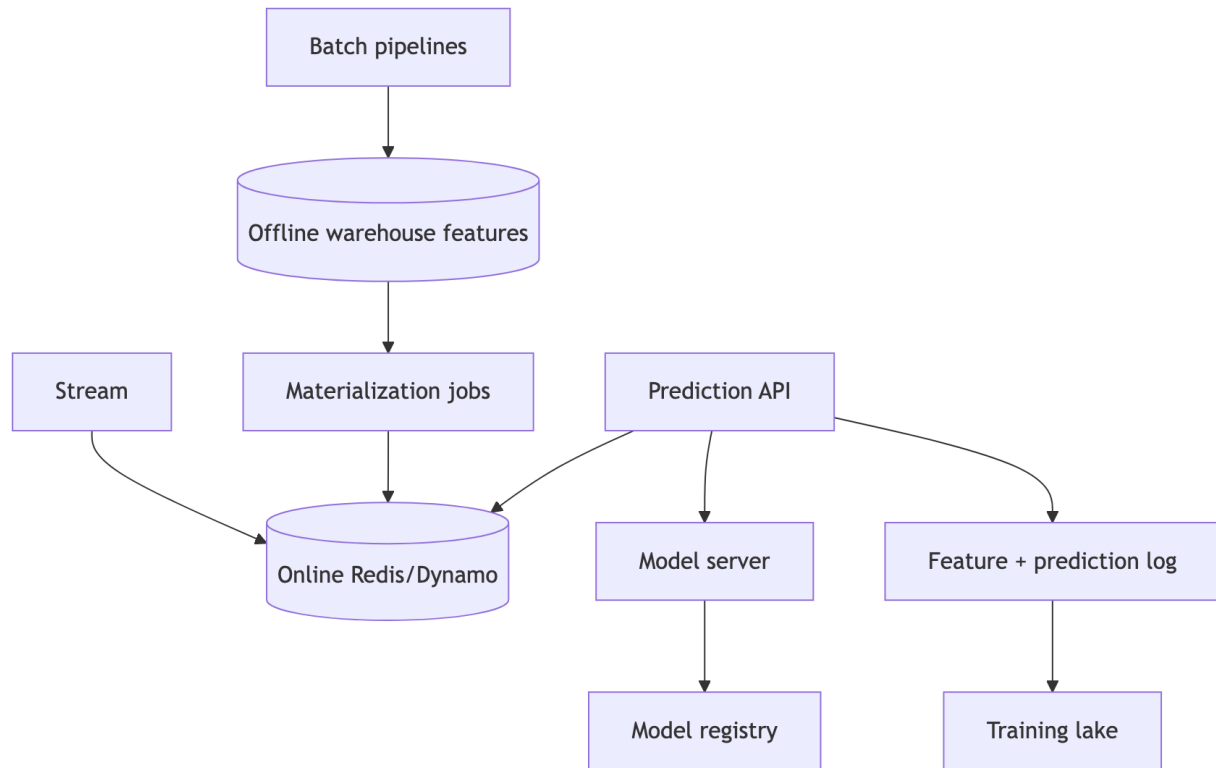


Figure 36: Diagram 36

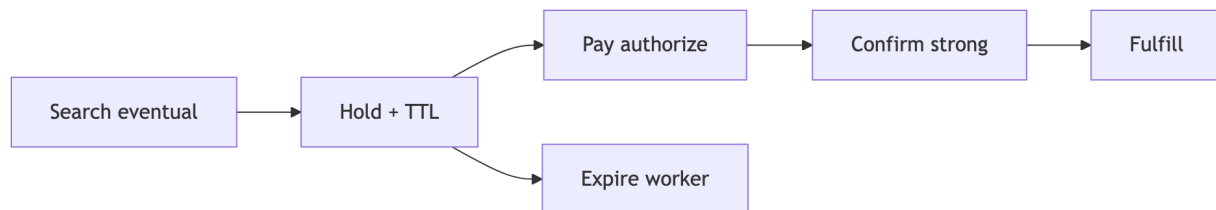


Figure 37: Diagram 37

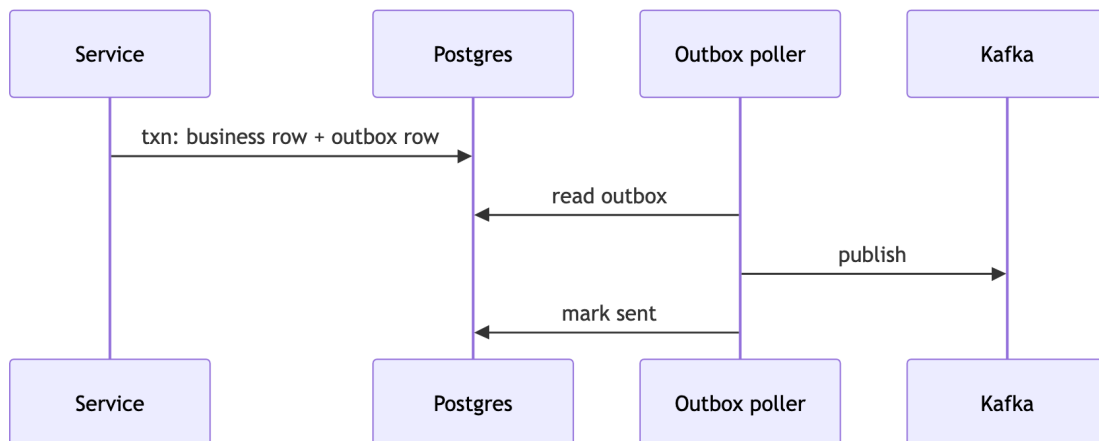


Figure 38: Diagram 38

Pattern	Use
Cache-aside	Pastebin reads, availability
Write-through	Rare; simpler consistency
TTL + jitter	Avoid stampedes
Singleflight	Hot key miss

17.4 When to pick stores

Data	Store
Bookings, users, money	Postgres
Hot keys / locks / GEO / rate limits	Redis
Spiky ingest	Kafka
Blobs	S3
Time series	TSDB
Vectors	Vector DB
Huge append messages	Cassandra/Dynamo

18. Capacity Estimation Workbook

Template

```

DAU = _
Actions/user/day = _
Daily requests = DAU × actions
Avg QPS = daily / 86400
Peak QPS    avg × 3..10
Storage/day = writes/day × bytes

```

Pastebin

```

5e6 writes/day × 8KB = 40GB/day
Reads 30× → ~1.7k avg QPS, design cache for 50k peak hot key

```

Metrics

```

50k samples/sec avg → Kafka + TSDB; rollups mandatory

```

Rides (city)

```

5k drivers × 1 Hz location = 5k QPS → Redis GEO
100 trips/sec peak → Postgres OK

```

RAG

10M docs × 20 chunks = 200M vectors
 64-byte dim compressed ~ ... size vector DB; embed async
 Query QPS 50 p95 < 2s with ANN + cache

19. Failure Modes Catalog

Failure	Detection	Mitigation
Payment OK, DB fail	webhook missing order	reconciler / outbox
Kafka lag spike	consumer lag metric	autoscale; shed non-critical
Cache stampede	origin QPS spike	singleflight + lock
Primary DB down	health check	failover; degrade reads
Duplicate POST	client retry	Idempotency-Key UNIQUE
Poison message	retry count	DLQ + alert
Hot partition	broker CPU	rehash key
ACL bug in RAG	audit samples	deny-by-default tests

20. Practice Rubric & Mocks**Rubric / 10**

Pts	Bar
2	Clarifying Qs + numeric NFRs
2	Diagram + 5 entities/classes
2	APIs with example JSON
3	Concurrency/failure sequence
1	Scale phase + trade-off close

Mock order (recommended)

1. Parking (LLD)
2. Car Rental (LLD)
3. Metrics (HLD)
4. Pastebin (HLD)
5. Elevator (OOD)
6. RAG (FDE story)

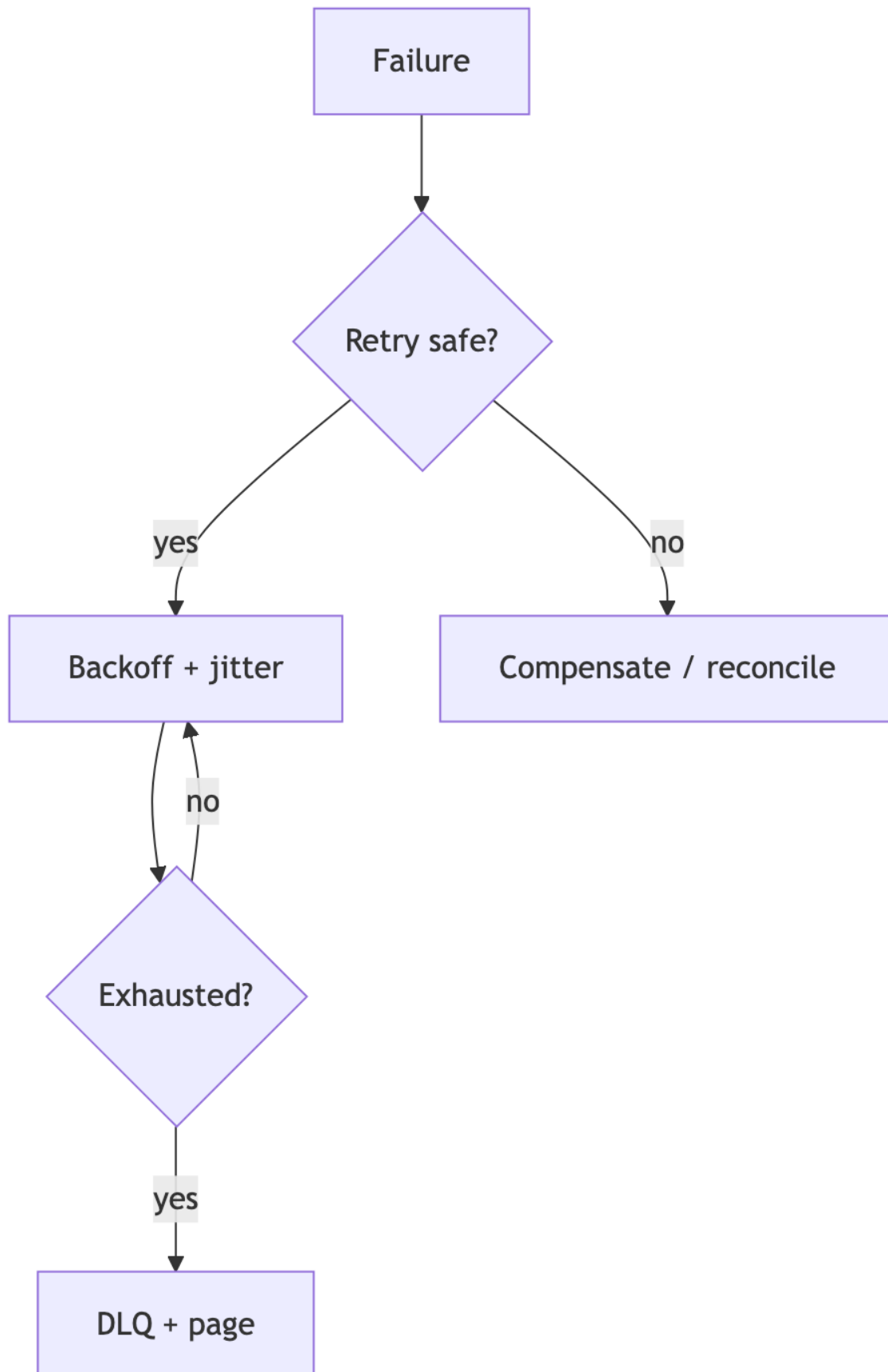


Figure 39: Diagram 39

7. Rides

8. Dropbox

Closing sentence bank

- “Reads can be eventual; assignments cannot.”
 - “Kafka protects storage from bursts.”
 - “Cache is never the source of truth for inventory.”
 - “Idempotency keys make mobile retries safe.”
 - “I’d ship v1 correct, then shard by lot/location.”
-

Whiteboard redraw kit

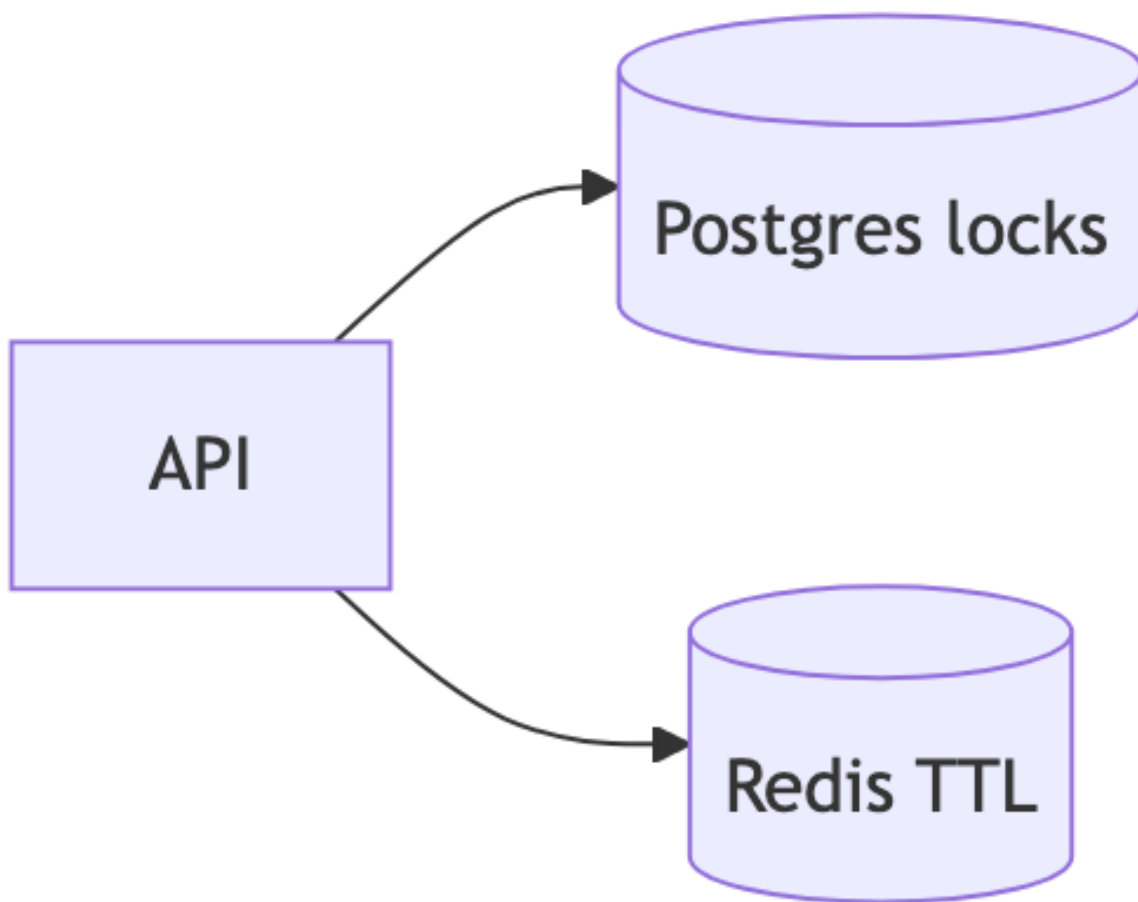


Figure 40: Diagram 40

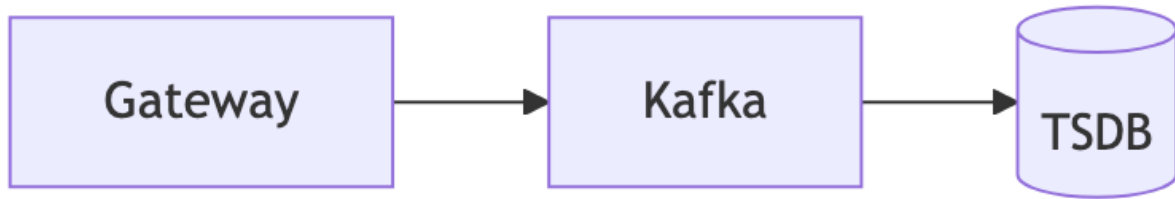


Figure 41: Diagram 41

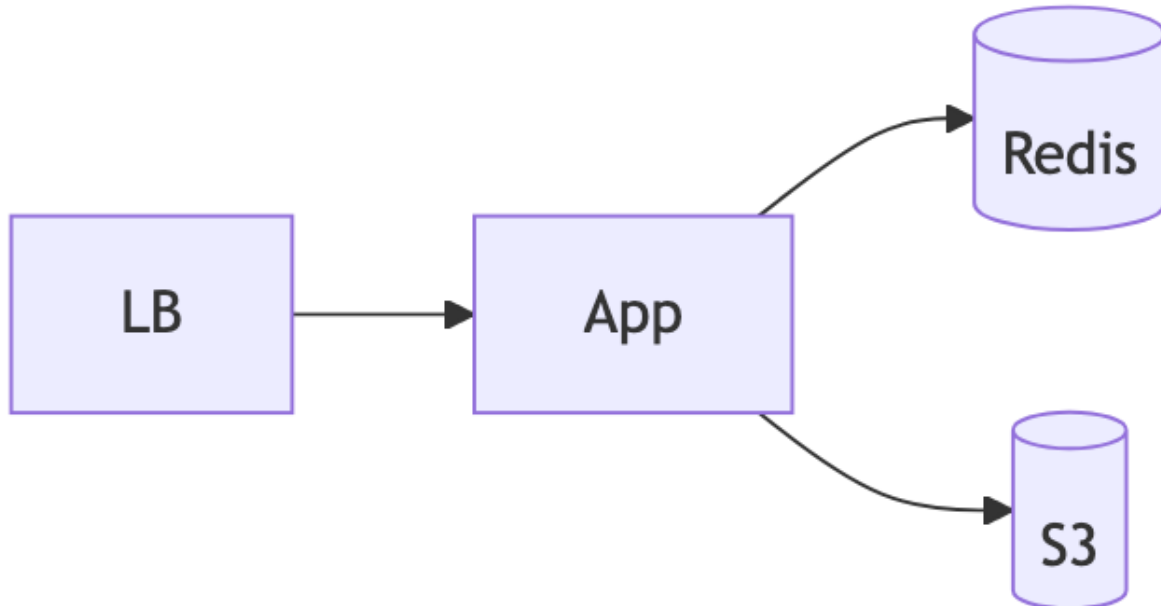


Figure 42: Diagram 42

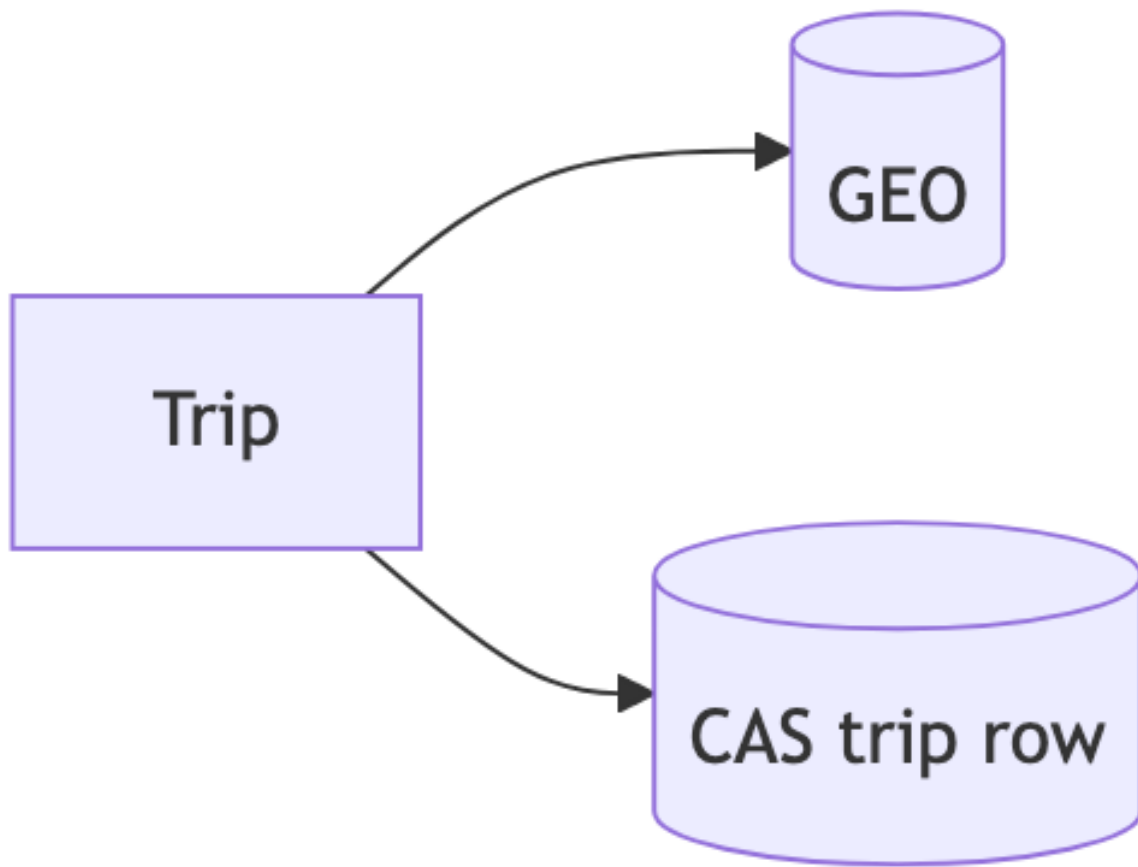


Figure 43: Diagram 43

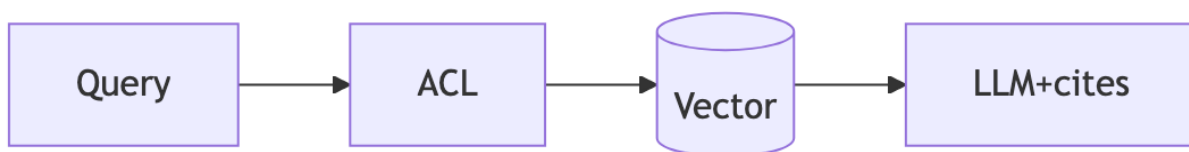


Figure 44: Diagram 44

Use this as a speaking script, not a memorized essay. Draw → narrate → deep-dive one invariant → stop.